



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

UNIT-2 OOP and Flutter Widgets

Dr. Annu Sharma

Department of Computer Applications

- **Object-oriented programming (OOP) is a programming method that uses objects and their interactions to design and program applications.**
- **It is one of the most popular programming paradigms and is used in many programming languages, such as Dart, Java, C++, Python, etc.**
- **In OOP, an object can be anything, such as a person, a bank account, a car, or a house.**
- **Each object has its attributes (or properties) and behavior (or methods). For example, a person object may have the attributes name, age and height, and the behavior walk and talk.**

Advantages of OOPs

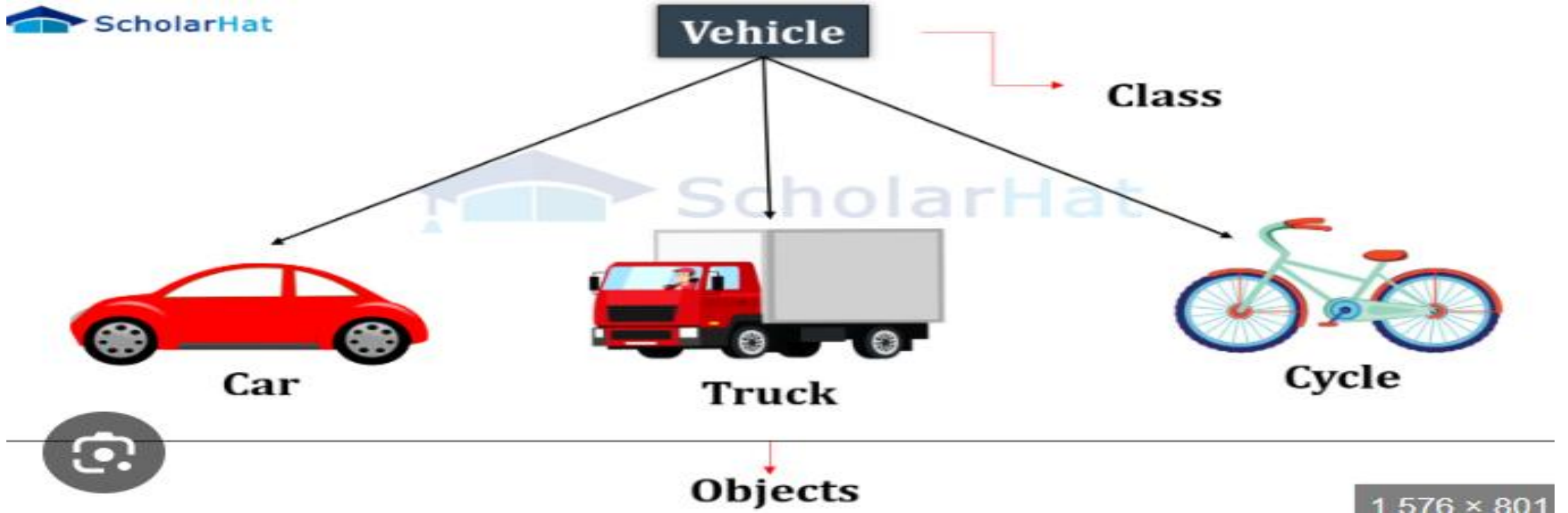
- It is easy to understand and use.
- It increases reusability and decreases complexity.
- The productivity of programmers increases.
- It makes the code easier to maintain, modify and debug.
- It promotes teamwork and collaboration.
- It reduces the repetition of code.

Features Of OOP

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

Features Of OOP

- Object



Key Points

- **Object Oriented Programming (OOP) is a programming paradigm that uses objects and their interactions to design and program applications.**
- **OOP is based on objects, which are data structures containing data and methods.**
- **OOP is a way of thinking about programming that differs from traditional procedural programming.**
- **OOP can make code more modular, flexible, and extensible.**
- **OOP can help you to understand better and solve problems.**

Class In Dart

In object-oriented programming, a class is a blueprint for creating objects. A class defines the properties and methods that an object will have. For example, a class called **Dog** might have properties like **breed**, **color** and methods like **bark**, **run**.

Declaring Class In Dart

You can declare a class in dart using the class keyword followed by **class** name and braces {}. It's a good habit to write class name in **PascalCase**. For example, **Employee**, **Student**, **QuizBrain**, etc.

Syntax

```
class ClassName {  
    // properties or fields  
    // methods or functions  
}
```

In the above syntax:

- The **class** keyword is used for defining the class.
- **ClassName** is the name of the class and must start with capital letter.
- Body of the class consists of **properties and functions**.
- **Properties** are used to store the data. It is also known as **fields or attributes**.
- Functions are used to perform the operations. It is also known as methods.

Example 1: Declaring A Class In Dart

```
class Animal {  
  String? name;  
  int? numberOfLegs;  
  int? lifeSpan;  
  void display() {  
    print("Animal name: $name.");  
    print("Number of Legs: $numberOfLegs.");  
    print("Life Span: $lifeSpan.");  
  }  
}
```

In this example , there is class **Animal** with three properties: **name**, **numberOfLegs**, and **lifeSpan**. The class also has a method called **display**, which prints out the values of the three properties.

Mobile App Development

Class Example-2



```
class Student {  
String? name;  
int? number;  
int? age;  
  
void display() {  
    print("student name: $name.");  
    print("Number: $number.");  
    print("Life Span: $age.");  
}  
}
```

```
void main(){  
    // Here std is object of class  
    Student.  
    Student std = Student();  
    std.name = "Arnav";  
    std.number = 4;  
    std.age = 10;  
    std.display();  
}
```

Function Call in Dart



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associat Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOP and Flutter Widgets

Object, Methods and Constructors

Dr. Annu Sharma

Department of Computer Applications

Dart Object

- Dart is object-oriented programming, and everything is treated as an object in Dart.
- An object is a variable or instance of the class used to access the class's properties.
- Objects have two features - **State and Behavior**.
- Programming objects are theoretically similar to the real-life objects; they also have state and behavior.
- An object is created from a template which is known as class.
- **The fields of the classes are stored as object states, whereas the method represents an object's behavior.**

Creating Class Objects in Dart

After creating the class, we can create an instance or object of that class w. The new keyword is used to declare class followed by the class name. general syntax of cr

Syntax for creating object

```
var object_name = new class_name(<constructor_arguments>);
```

```
class Student {  
String? name;  
int? number;  
int? age;  
  
void display() {  
    print("student name: $name.");  
    print("Number: $number.");  
    print("Life Span: $age.");  
}  
}
```

```
void main(){  
    // Here std is object of class Student.  
    Student std = Student();  
    std.name = "Arnav";  
    std.number = 4;  
    std.age = 10;  
    std.display();  
}Function Call in Dart
```

Assessing Instance Variable and Function Features Of OOP

- After creating an object, we can access the fields and methods of the class.
- The class property name is separated by the (.) operator with the instance name.

syntax

objectName.propName or objectName.methodName()

Assessing Instance Variable and Function Features Of OOP

- Example:

```
std.name = "Arnav";  
std.number = 4;  
std.age = 10;  
std.display();
```

Benefit of Objects

- **Modularity:** The source code of an object can be maintained individually and can hide from the other object's source code.
- **Data - hiding:** Using oops programming, the details of the internal functionality of code are hidden from the others.
- **Reusability** - We don't need to write the same code again and again. We can use the object of class multiple times in our program.
- **Pluggability and debugging easy** - If any object is creating a problem in our program, and then we can replace it in our program and plug the new object as its replacement. The oops code can be easy to debug.

Key Points

- The class is declared using the class keyword.
- The class is a blueprint for creating objects.
- The class body consists of properties and methods.
- The properties are also known as fields, attributes, or data members.
- The methods are also known as behaviors, or member functions.

Challenge

Create a class Book with three properties: name, author, and price. Also, create a method called display, which prints out the values of the three properties.

Dart Constructors

- A constructor is a special function of the class that is responsible for initializing the variables of the class.
- Dart defines a constructor with the same name as that of the class.
- The constructor is used to initialize an object when it is created.
- A constructor is a function and hence can be parameterized.
- Constructors cannot have a return type.
- If you don't declare a constructor, a default no-argument constructor is provided for you.

Dart Constructors

Syntax

```
class ClassName {  
    ClassName() {  
    }  
}
```

Example

```
Class Student{  
    Student(){  
    }  
}
```

Remember the following two rules while creating a constructor.

- **The constructor name should be same as the class name.**
- **The constructor doesn't have explicit return type.**

Dart Constructors Example

```
void main() {  
    // Creating an object  
    Student std = new Student("shrishti",26);  
}  
class Student{  
    // Declaring a construtor  
    Student(String str, int age)  
    {  
        print("The name is: ${str}");  
        print("The age is: ${age}");  
    }  
}
```

Types of Constructor

There are three types of constructors in Dart

- Default Constructor or no-arg Constructor
- Parameter Constructor
- Named Constructor



THANK YOU

Dr. Annu Sharma

Associate Professor

Department of Computer Applications



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOP and Flutter Widgets

Classes - Constructors Types, Setters & Getters

Dr. Annu Sharma

Department of Computer Applications

Types of Constructor

There are three types of constructors in Dart

- Default Constructor or no-arg Constructor
- Parameter Constructor
- Named Constructor

Default Constructor or no-arg Constructor

- A Constructor which has no parameter is called default constructor or no-arg constructor.
- It is automatically created (with no argument) by Dart compiler if we don't declare in the class.
- The Dart compiler ignores the default constructor if we create a constructor with argument or no argument.

Syntax:

```
class ClassName {  
    ClassName() {  
        // constructor body  
    }  
}
```

Default Constructor or no-arg Constructor Example

```
void main() {  
    // Call constructor automatically when we creates an object  
    Student std = new Student();  
}  
  
class Student{  
    // Declaring a construtor  
    Student()  
{  
    print("The example of the default constructor");  
}  
}
```

Parameterized Constructor

- We can also pass the parameters to a constructor that type of constructor is called parameterized constructor.
- It is used to initialize instance variables.
- Sometimes, we need a constructor which accepts single or multiple parameters.
- The parameterized constructors are mainly used to initialize instance variable with own values.

Syntax:

```
class ClassName {  
    ClassName(parameter_list)  
    // constructor body }
```

Parameterized Constructor Example

```
void main() {  
    // Creating an object  
    Student std = new Student("Shashank",26);  
}  
class Student{  
    // Declaring a parameterized constructor  
    Student(String str, int age){  
        print("The name is: ${str}");  
        print("The age is: ${age}");  
    }  
}
```

Dart Constructors

Named Constructors

The named constructors are used to declare the multiple constructors in single class. The syntax is given below.

Syntax:

```
className.constructor_name(param_list)
```

Named Constructor Example

```
void main() {  
    // Creating an object  
    Student std1 = new Student(); //  
    object for Default constructor  
    Student std2 = new  
    Student.namedConst("Computer  
    Science"); // object for parameterized  
    constructor  
}
```

```
class Student{  
    // Declaring a construtor  
    Student(){  
        print("The example of the named  
        constructor");  
    }  
    // Second constructor  
    Student.namedConst(String branch){  
        print("The branch is: ${branch}");  
    }  
}
```

This keyword

- This keyword is used to refer the current class object.
- It indicates the current instance of the class, methods, or constructor.
- It can be also used to call the current class methods or constructors. It eliminates the uncertainty between class attributes and the parameter names are the same.
- If we declare the class attributes same as the parameter name, that situation will create ambiguity in the program, then the this keyword can remove the ambiguity by prefixing the class attributes.
- It can be passed as an argument in the class method or constructor.

Dart Getters and Setters

1. Getter and setter methods are the class methods used to manipulate the data of the class fields.
2. Getter is used to read or get the data of the class field whereas
3. Setter is used to set the data of the class field to some variable.setter method is used to set or initialize respective class fields
4. By default, all classes are associated with getter and setter method. However, we can override the default methods by defining getter and setter method explicitly.

Getter Method in Dart

It is used to retrieve a particular class field and save it in a variable.

All classes have a default getter method but it can be overridden explicitly

The getter method can be defined using the **get keyword** as:

```
return_type get field_name{  
    ...  
}
```

Note: we have to define a return type but there is no need to define parameters in the above method.

Setter Method in Dart

It is used to set the data inside a variable received from the getter method. All classes have a default setter method but it can be overridden explicitly. The setter method can be defined using the **set keyword** as:

```
set field_name{  
    ...  
}
```

Mobile App Development

Dart Example Getter and Setter

```
class Student {  
  String _name;  
  int _age;  
  // Constructor  
  Student(this._name, this._age);  
  // Getter for name  
  String get name => _name;  
  // Setter for name  
  set name(String name) {  
    if (name.isEmpty) {  
      _name = name;  
    } else {  
      print('Name cannot be empty');  
    }  
  }  
}
```

```
// Getter for age  
int get age => _age;  
// Setter for age  
set age(int age) {  
  if (age > 0) {  
    _age = age;  
  } else {  
    print('Age must be positive');  
  }  
}
```

Dart Example Getter and Setter contd.

```
void main() {  
    Student student = Student('Vyshak', 20);  
  
    // Using getters  
    print('Name: ${student.name}');  
    print('Age: ${student.age}');  
  
    // Using setters  
    student.name = 'Vachna';  
    student.age = 19;  
  
    print('Updated Name: ${student.name}');  
    print('Updated Age: ${student.age}');  
}
```

Challenge

Create a class Rectangle with three properties: width and height. Using getter and setter methods set and get the value width and height and create a method to calculate the Area of Rectangle using these values.



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOP and Flutter Widgets Inheritance in Dart

Dr. Annu Sharma

Department of Computer Applications

INHERITANCE IN DART

- Inheritance is a major component of a programming paradigm known as **OOPS** (Object-Oriented Programming).
- Inheritance in dart is defined as the process in which one class **derive the properties and characteristics** of another class.
- It is helpful as it provides an ability with which **we can create a new class from an existing class**.
- With the help of Inheritance, one class can make use of **all the properties and characteristics** of another class.

In general, two classes are required for inheritance and these mainly are –

- **Parent class** - A class that is inherited by the other class is known as the parent class. Sometimes, we also refer it to as the base class.
- **Child class** - A class that inherits the properties and characteristics of the parent class is known as child class.

Dart Inheritance Syntax

Syntax

```
class ParentClass {  
    // Parent class code  
}  
  
class ChildClass extends ParentClass {  
    // Child class code  
}
```

In this syntax, **ParentClass** is the super class and **ChildClass** is the sub class. The **ChildClass** inherits the properties and methods of the **ParentClass**.

Dart Inheritance Syntax

```
class Person {  
  // Properties  
  String? name;  
  int? age;  
  
  // Method  
  void display() {  
    print("Name: $name");  
    print("Age: $age");  
  }  
}
```

```
// Here In student class, we are extending  
the properties and methods of the Person  
class Student extends Person {  
  // Fields  
  String? schoolName;  
  String? schoolAddress;  
  // Method  
  void displaySchoolInfo() {  
    print("School Name: $schoolName");  
    print("School Address:  
$schoolAddress");  
  }  
}
```

Dart Inheritance Example

```
void main() {  
  // Creating an object of the Student class  
  var student = Student();  
  student.name = "varun";  
  student.age = 20;  
  student.schoolName = "PES UNIVERSITY";  
  student.schoolAddress = "BENGALURU";  
  student.display();  
  student.displaySchoolInfo();  
}
```

Name: varun

Age: 20

School Name: PES UNIVERSITY

School Address: BENGALURU

Advantages Of Inheritance In Dart

- It promotes reusability of the code and reduces redundant code.
- It helps to design a program in a better way.
- It makes code simpler, cleaner and saves time and money on maintenance.
- It facilitates the creation of class libraries.
- It can be used to enforce standard interface to all children classes.

Types of Inheritance in Dart

There are different types of inheritance that are possible in Dart. Mainly these are –

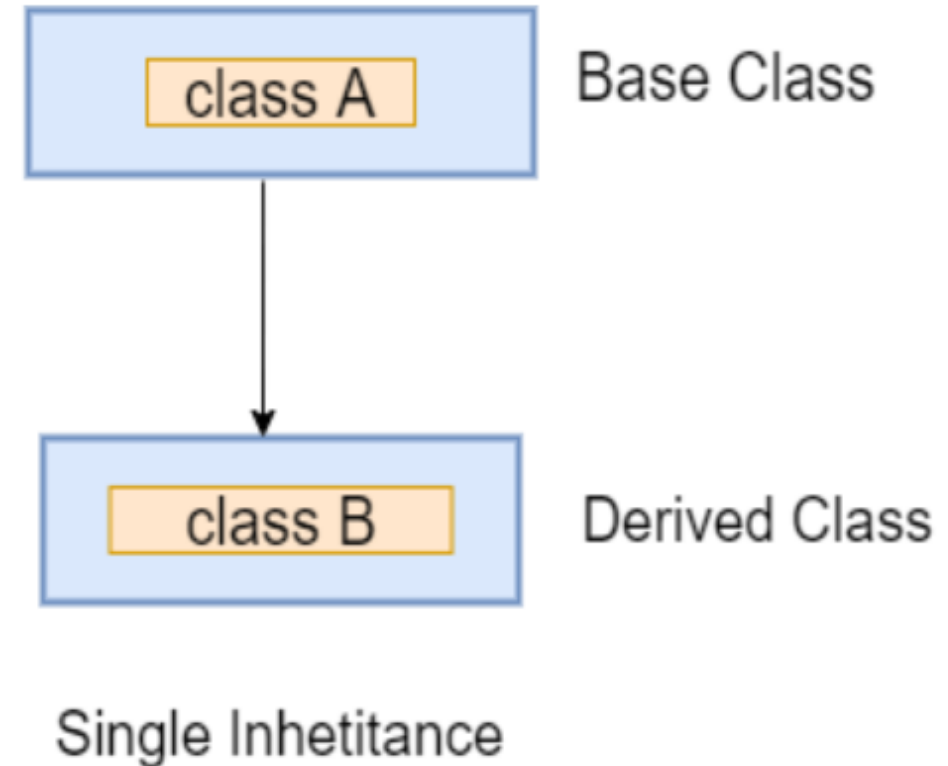
- Single level inheritance
- Multi-level inheritance
- Hierarchal inheritance

Types of Inheritance in Dart

Single Level Inheritance

In the single inheritance, a class is inherited by a single class or subclass is inherited by one parent class. In the following example, we create Person which inherits Human class.

Dart Inheritance



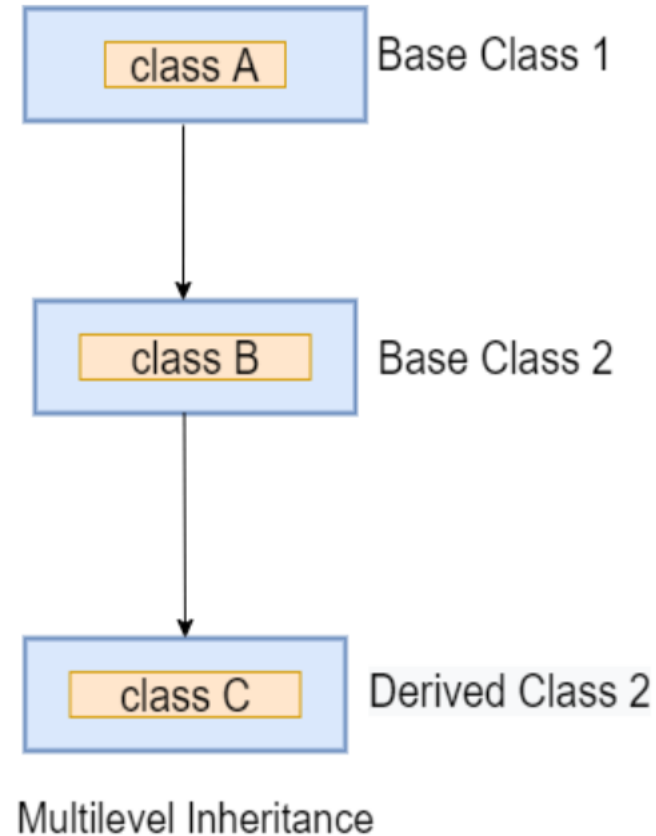
Single Level Inheritance

```
class Bird{  
    void fly()  
    {  
        print("The bird can fly");  
    }  
}  
  
// Inherits the super class  
class Parrot extends Bird{  
    //child class function  
    void speak(){  
        print("The parrot can speak");  
    }  
}
```

```
void main() {  
    // Creating object of the child class  
    Parrot p=new Parrot();  
    p.speak();  
    p.fly();  
}
```

MultiLevel Inheritance

In the multiple inheritance, a subclass is inherited by another subclass or creates the chaining of inheritance. Let's understand the following example.



MultiLevel Inheritance

```
class Bird{
    void fly()
    {
        print("The bird can fly");
    }
}

// Inherits the super class
class Parrot extends Bird{
    void speak(){
        print("The parrot can speak");
    }
}
}
```

```
// Inherits the Parrot base class
class Eagle extends Parrot {
    void vision(){
        print("The eagle has a sharp vision");
    }
}

void main() {
    // Creating object of the child class
    Eagle e=new Eagle();
    e.speak();
    e.fly();
    e.vision();
}
```

Challenge

Create class named Person with two properties name and age. sub class named Doctor with properties listofdegrees and hospitalname. Another subclass named Specialist with property specialization. The sub class has a method display to display the values of the properties.



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associat Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOP and Flutter Widgets

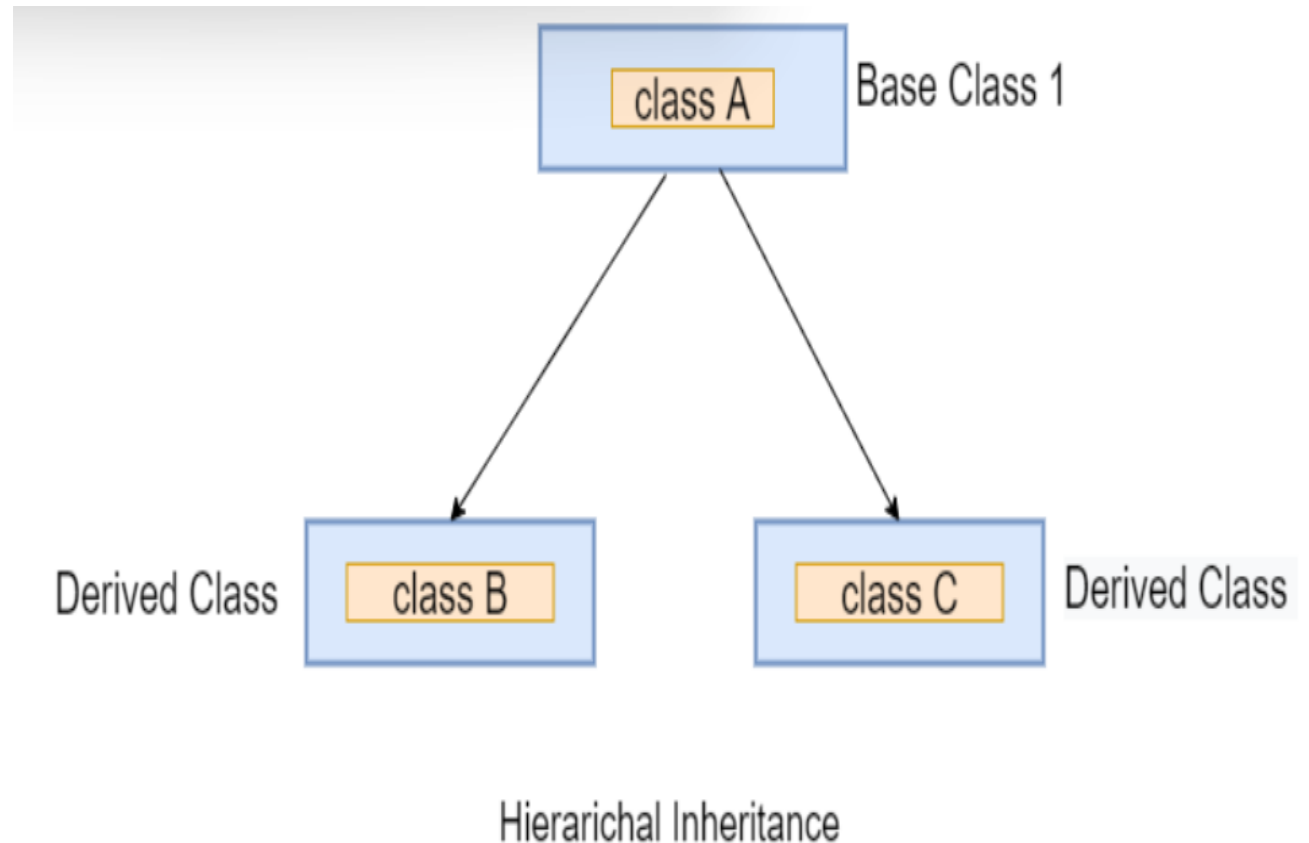
Abstract Class

Dr. Annu Sharma

Department of Computer Applications

Hierarichal Inheritance

In the hierarchical inheritance, two or more classes inherit a single class. In the following example, the two-child classes class B and class C inherit the class A.



Hierarichal Inheritance Example

```
// Parent Class
class Person {
  void dispName(String name) {
    print(name);
  }
  void dispAge(int age) {
    print(age);
  }
}
class Prithvi extends Person {
  void dispBranch(String nationality) {
    print(nationality);
  }
}
```

```
class Yashwin extends Person {
  void result(String result){
    print(result);
  }
}
void main() {
  // Creating Object of James class
  Yashwin y = new yashwin();
  y.dispName("James");
  y.dispAge(24);
  y.result("Passed");
  // Creating Object of Prithvi class
  Prithvi p = new Prithvi();
  p.dispName("Peter");
  p.dispAge(21);
  p.dispBranch("Computer Science");
}
```

Abstract Class in Dart

Abstract classes are classes that cannot be initialized. It is used to define the behavior of a class that can be inherited by other classes. An abstract class is declared using the keyword `abstract`.

Syntax:

```
abstract class ClassName {  
    //Body of abstract class  
    method1();  
    method2();  
}
```

Abstract Class in Dart

- Abstract classes are the classes in Dart that has one or more abstract method. Abstraction is a part of the data encapsulation where the actual internal working of the function hides from the users.
- They interact only with external functionality.
- We can declare the abstract class by using the **abstract keyword**.
- There is a possibility that an abstract class may or may not have abstract methods.

Abstract Class

1. We cannot create the instance of an abstract class that means it can't be instantiated.
2. It can only be extended by the subclass, and the subclass must be provided the implantation to the abstract methods which are present in the present class.
3. Then it is necessary to declare abstract subclass.

Abstract Methods

- Abstract methods are those methods, which are declared without implementation.
- The concrete methods or normal methods are declared with implementation.
- An abstract class can contain both types of methods, but a normal class is not allowed to have abstract methods.

Abstract Method in Dart

An abstract method is a method that is declared without an implementation. It is declared with a semicolon (;) instead of a method body.

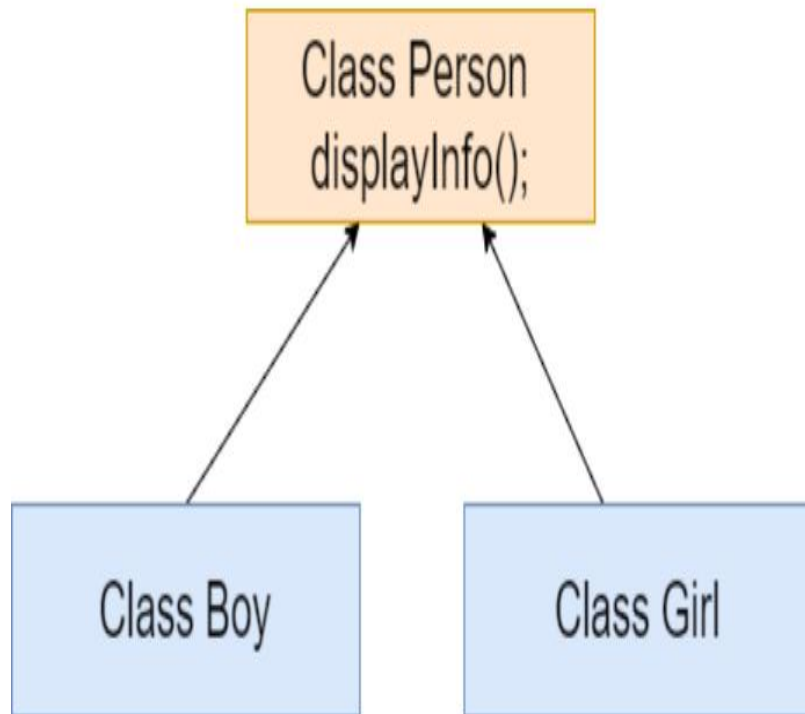
Syntax

```
abstract class ClassName {  
    //Body of abstract class  
    method1();  
    method2();  
}
```

Rules for Abstract classes:

1. An abstract class can have an abstract method (method without implementation), or not.
2. If there is at least one abstract method, then the class must be declared abstract.
3. The object of the abstract class cannot be created, but it can be extended.
4. An abstract keyword is used to declare the abstract class.
5. An abstract class can also include normal or concrete (method with the body) methods.
6. All abstract methods of parent class must be implemented in the subclass.

Usage of Abstract class



we have a class **Person** that has method **displayInfo()**, and we have to sub classes of it **Boy** and **Girl**.

Each of the person information varies from the other person, so there is no benefit to implementing the **displayInfo()** in the parent class.

Because every subclass must override the parent class method by provides its own implementation. Thus, we can force the subclass to provide implementation to that method, so that is the benefit to make method abstract.

Abstract Class Example 1

```
abstract class Person {  
  //declaring abstract method  
  void displayInfo(); //abstract method  
}  
class Boy extends Person  
{  
  // Overriding method  
  void displayInfo() {  
    print("My name is Ashok");  
  }  
}
```

class Girl extends Person

```
{  
  // Overriding method  
  void displayInfo() {  
    print("My name is Garima");  
  } }  
  void main() {  
    Boy b = new Boy(); // Creating Object of Boy class  
    Girl g = new Girl(); // Creating Object of Girl class  
    b.displayInfo();  
    g.displayInfo();  
  }
```

```
abstract class Shape {  
  int dim1, dim2;  
  // Constructor  
  Shape(this.dim1, this.dim2);  
  // Abstract method  
  void area();  
}  
class Rectangle extends Shape {  
  // Constructor  
  Rectangle(int dim1, int dim2) : super(dim1, dim2);  
  // Implementation of area()  
  @override  
  void area() {  
    print('The area of the rectangle is ${dim1 * dim2}');  
  }  
}
```

```
class Triangle extends Shape {  
  // Constructor  
  Triangle(int dim1, int dim2) : super(dim1, dim2);  
  // Implementation of area()  
  @override  
  void area() {  
    print('The area of the triangle is ${0.5 * dim1 *  
dim2}');  
  }  
}  
void main() {  
  Rectangle rectangle = Rectangle(10, 20);  
  rectangle.area();  
  Triangle triangle = Triangle(10, 20);  
  triangle.area();  
}
```

Challenge

- 1: Shows how abstract classes can be used to define shapes like circles and rectangles.**
- 2: Demonstrates the use of abstract classes in a vehicle hierarchy.**
- 3: Illustrates an abstract payment system.**
- 4: Introduces a logging system using an abstract class.**



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOP and Flutter Widgets DART LIBRARIES

Dr. Annu Sharma

Department of Computer Applications

DART Libraries

- Dart Project Structure and Dart Libraries
- Understanding Flutter, Flutter Framework,
- Installing Android Studio, Configure Flutter SDK, Install and Configure SDK,
Installing and Configuring Flutter SDK
- Setup an Android Virtual Device, Run a Flutter App
- Android Studio Sugar and Spice, Run your Flutter App
- Emulator Debug Mode, Creating a Simple Flutter App.

Dart Project Structure and Dart Libraries

- A library in a programming language represents a collection of routines (set of programming instructions).
- Dart has a set of built-in libraries that are useful to store routines that are frequently used.
- A Dart library comprises of a set of classes, constants, functions, typedefs, properties, and exceptions.

Importing a library

- Importing makes the components in a library available to the caller code.
- The import keyword is used to achieve the same.
- A dart file can have multiple import statements.
- Built in Dart library URIs use the dart: scheme to refer to a library.
- Other libraries can use a file system path or the package: scheme to specify its URI.
- Libraries provided by a package manager such as the pub tool uses the *package: scheme*.

- In the Dart programming language, **built-in libraries** (also called **core libraries**) are accessed using a special URI scheme that starts with dart :
- **URI** stands for **Uniform Resource Identifier**.
- It's a string used to identify a resource, similar to how <https://example.com> identifies a webpage.
- In Dart, URIs are used to **import libraries**.

Importing a library

The syntax for importing a library in Dart is given below –

```
import 'URI'
```

Consider the following code snippet –

```
import 'dart:io'
```

```
import 'package:lib1/libfile.dart'
```

URI can point to:

- A built-in Dart library
- A file in your project
- An external package (like from pub.dev)

Importing a library

Some commonly used libraries in Dart –

Sr.No	Library & Description
1	<code>dart:io</code> File, socket, HTTP, and other I/O support for server applications. This library does not work in browser-based applications. This library is imported by default.
2	<code>dart:core</code> Built-in types, collections, and other core functionality for every Dart program. This library is automatically imported.

Importing a library

Some commonly used libraries in Dart –

Sr.No	Library & Description
3	<code>dart: math</code> Mathematical constants and functions, plus a random number generator.
4	<code>dart: convert</code> Encoders and decoders for converting between different data representations, including JSON and UTF-8.
5	<code>dart: typed_data</code> Lists that efficiently handle fixed sized data (for example,

Example - Importing and using a Library

```
import 'dart:math'; // Importing built-in library
```

```
void main() {  
    print("Square root of 36 is: ${sqrt(36)}");  
}
```

Output:

Square root of 25 is: 5.0

Explanation:

In the above code, we imported the **built-in library 'dart:math'**. It provides the many built-in mathematical function, here we used the `sqrt()` function with number. It takes a number as an argument that we want to find its square root of. We passed an integer number 25 in `sqrt()` function, and it returned an output as 5.

DART Libraries

- i. **lib/**: The main directory for your Dart code. This folder contains your Dart files, typically starting with **main.dart**.
- ii. **test/**: This folder contains unit tests for your app.
- iii. **android/ and ios/**: These folders hold the native Android and iOS files required to run your Flutter app on respective platforms.
- iv. **pubspec.yaml**: The configuration file where you define dependencies, assets, and other project metadata.
- v. **build/**: A generated directory for compiled files.



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOP and Flutter Widgets

Understanding Flutter, Flutter Framework

Dr. Annu Sharma

Department of Computer Applications

A. Understanding Flutter and the Flutter Framework

- Flutter is an open-source UI software development kit (SDK) created by Google.
- It allows developers to create natively compiled applications for mobile (iOS, Android), web, and desktop from a single codebase.

The Flutter Framework includes:

- **Widgets:** The building blocks of a Flutter app. They are immutable and describe part of the user interface. Widgets can be stateless or stateful.
- **Rendering Engine:** Flutter uses Skia to render widgets. It allows drawing to the screen in a performant way. Skia serves as the graphics engine for Google Chrome and ChromeOS, Android, Flutter, and many other products.
- **Platform Channels:** These enable Dart to communicate with native (Android and iOS) code.

Flutter supported IDE's

- Visual Studio Code
 - Lightweight, fast, easy to set up
- Android Studio
- IntelliJ IDEA

A. Configuring Flutter SDK

1. After installing Android Studio, download the Flutter SDK from the [Flutter website] (<https://flutter.dev/docs/get-started/install>).
2. Extract the Flutter SDK to a desired location.
3. Update your system's environment variables to include Flutter's `bin` directory. This allows the Flutter command to be available globally.

Mobile App Development

Understanding Flutter, Flutter Framework



Flutter SDK

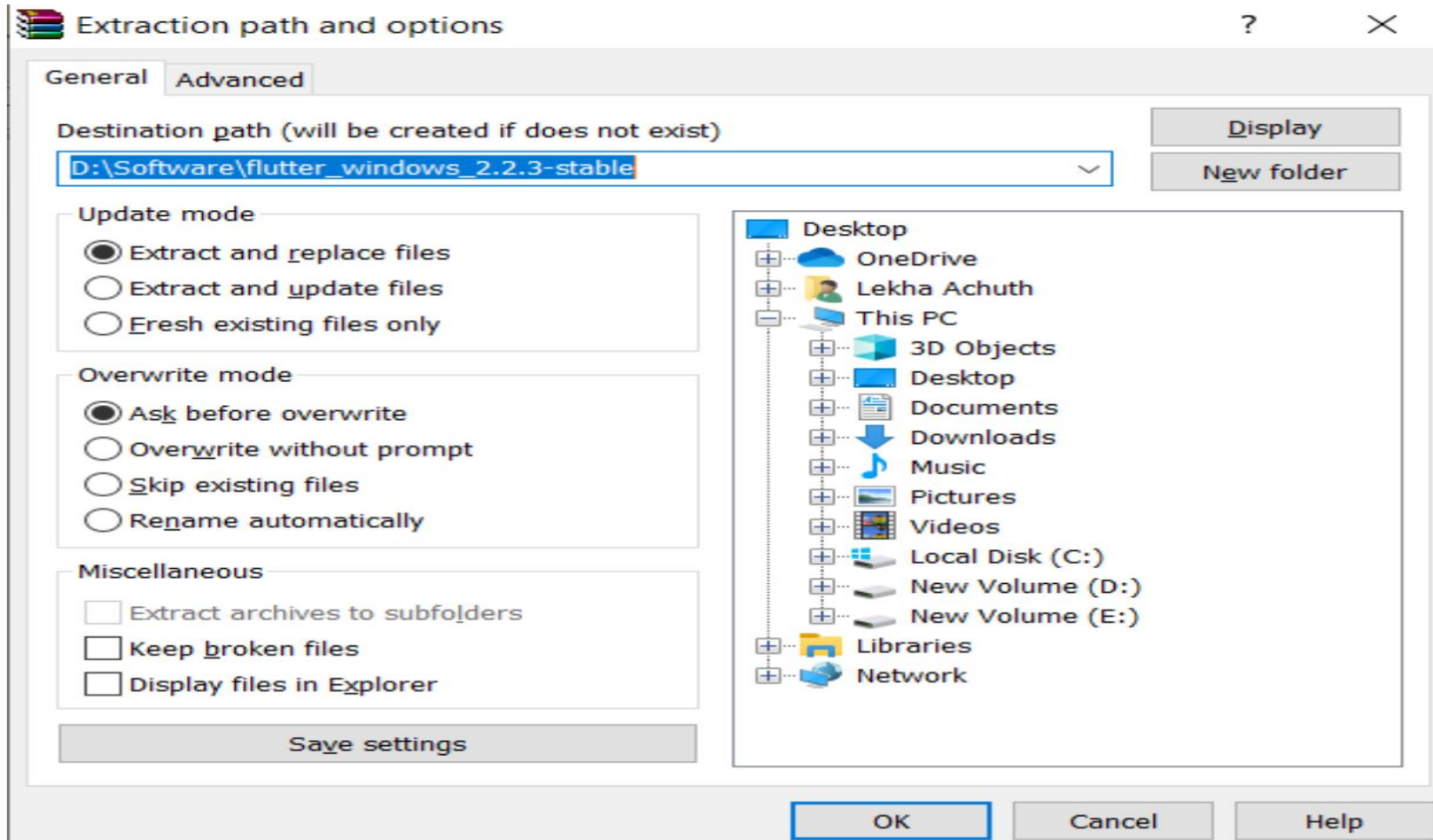
IT includes the Flutter compiler, project creator, device manager, test runner, and tools that diagnose– and even. correct– problems with the Flutter configuration.

A screenshot of the Flutter website's documentation page for installing the Flutter SDK on Windows. The browser address bar shows "flutter.dev/docs/get-started/install/windows". The page has a navigation bar with "Flutter" logo, "Docs", "Showcase", "Community", and a "Get started" button. A left sidebar lists "Get started" steps: 1. Install, 2. Set up an editor, 3. Test drive, 4. Write your first app, 5. Learn more, and a "From another platform?" section with links for Android, iOS, React Native, web, and Xamarin.Forms. The main content area is titled "Get the Flutter SDK" and lists two steps: 1. Download the installation bundle "flutter_windows_2.2.3-stable.zip" and 2. Extract the zip file and place the contained "flutter" in the desired installation location. A yellow warning box states: "Warning: Do not install Flutter in a directory like C:\Program Files\ that requires elevated privileges." Below this, it says: "If you don't want to install a fixed version of the installation bundle, you can skip steps 1 and 2. Instead, get the source code from the Flutter repo on GitHub, and change branches or tags as needed. For example:". A right sidebar titled "Contents" lists links for System requirements, Get the Flutter SDK, Update your path, Run flutter doctor, Android setup, Install Android Studio, Set up your Android device, Set up the Android emulator, Agree to Android Licenses, Windows setup, Additional Windows requirements, Enable desktop support, Web setup, and Next step.

Mobile App Development

Understanding Flutter, Flutter Framework

Flutter SDK

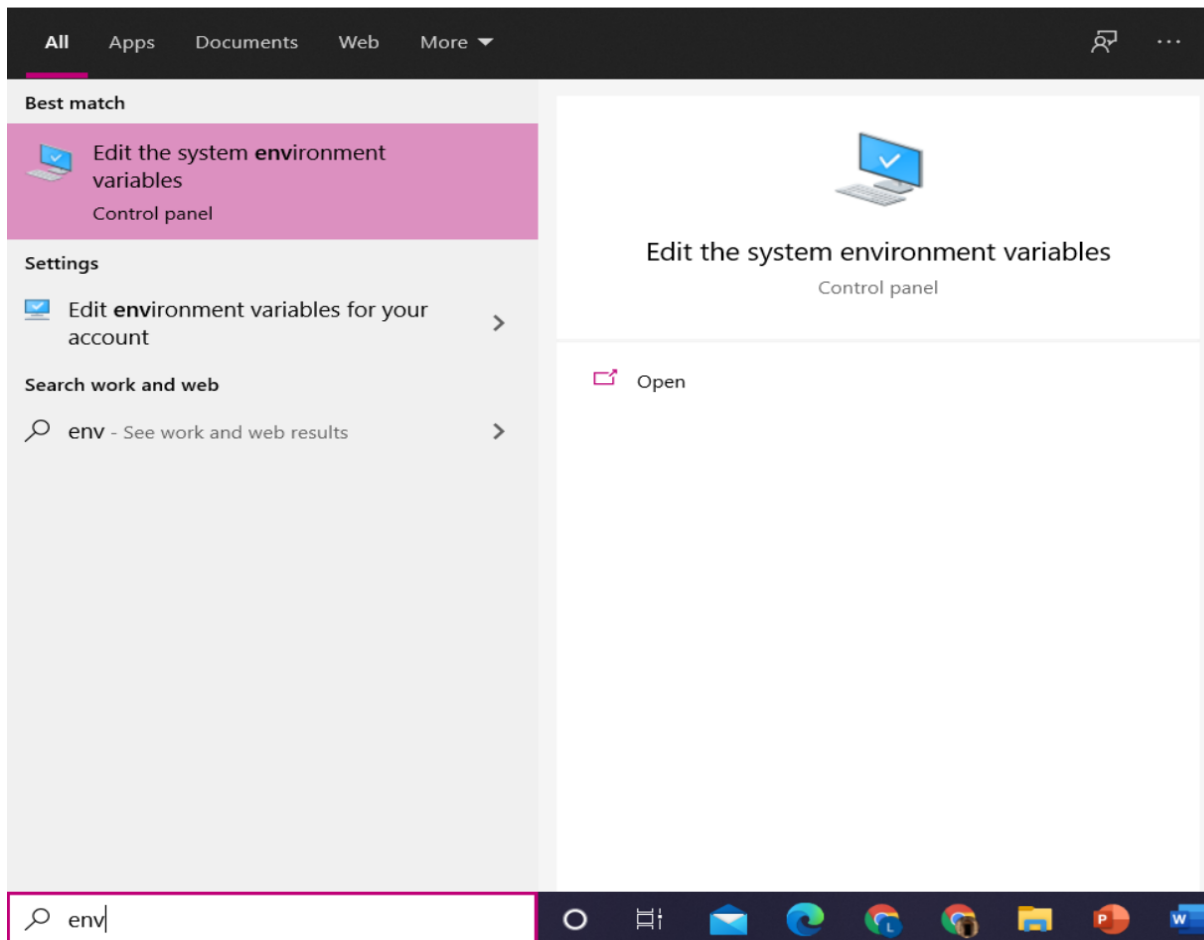


Mobile App Development

Understanding Flutter, Flutter Framework

Edit System variables

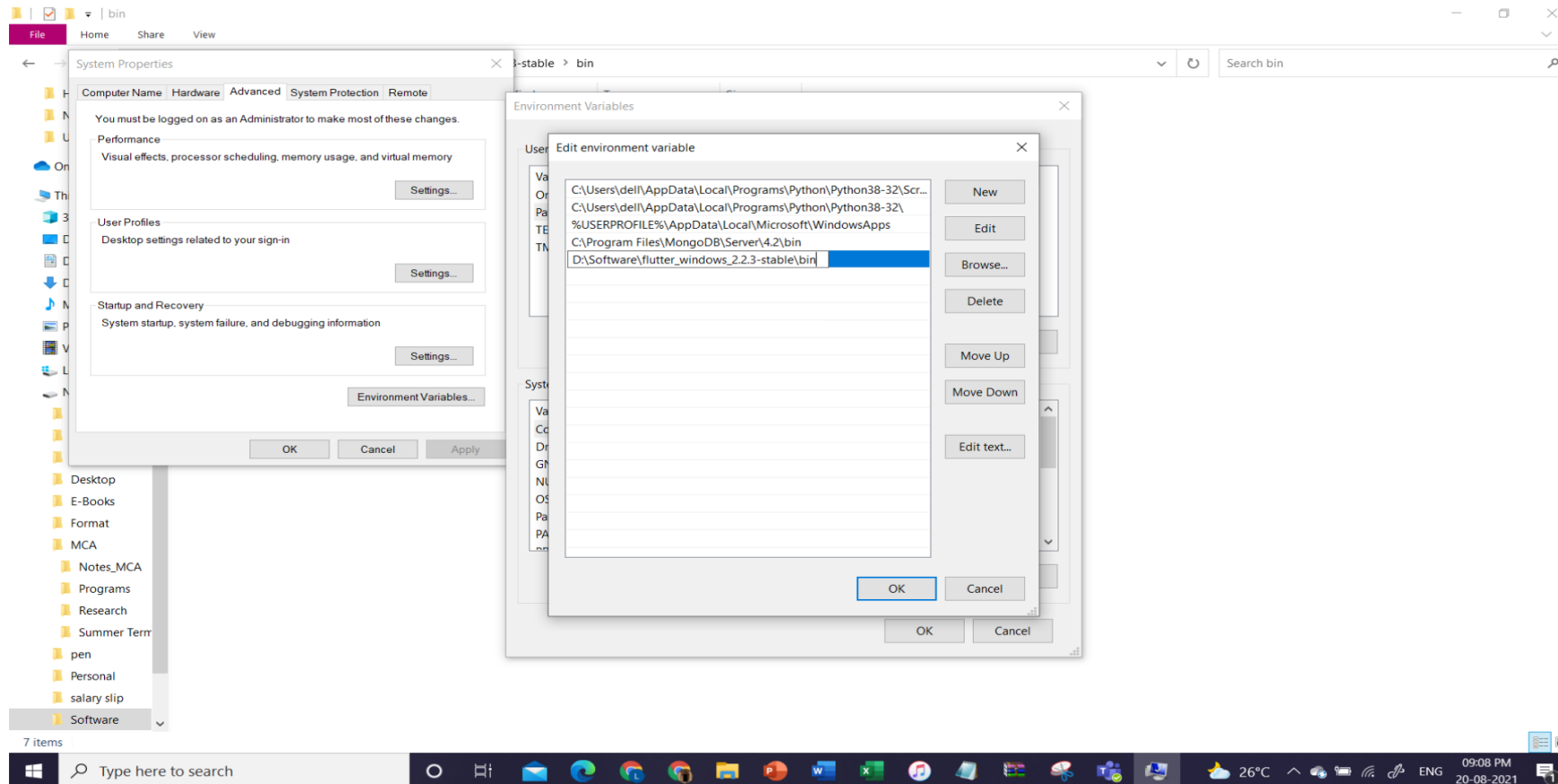
Update the path



Mobile App Development

Understanding Flutter, Flutter Framework

Edit System Environment variables: Update Path



Mobile App Development

Understanding Flutter, Flutter Framework



Checking the path setup

```
C:\Users\de11>where flutter dart
D:\Software\flutter_windows_2.2.3-stable\bin\flutter
D:\Software\flutter_windows_2.2.3-stable\bin\flutter.bat
D:\Software\flutter_windows_2.2.3-stable\bin\dart
D:\Software\flutter_windows_2.2.3-stable\bin\dart.bat
```

A. Installing and Configuring Flutter SDK

In addition to configuring the Flutter SDK:

1. Install **Dart SDK** (included with Flutter).
2. Set up an editor like **Android Studio**
3. Install the **Flutter and Dart plugins** from the plugins marketplace.

After installation, check the setup by running the following command in the terminal:

```
flutter doctor
```

This will provide feedback on the state of your Flutter installation and guide you through any missing dependencies.

Mobile App Development

Understanding Flutter, Flutter Framework



Run Flutter Doctor

```
C:\Users\dell>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 2.2.3, on Microsoft Windows [Version 10.0.19042.1165],
    locale en-IN)
[!] Android toolchain - develop for Android devices (Android SDK version 31.0.0)
    ✗ Android licenses not accepted.  To resolve this, run: flutter doctor
      --android-licenses
[✓] Chrome - develop for the web
[?] Android Studio (the doctor check crashed)
    ✗ Due to an error, the doctor check did not complete. If the error message below is
      not helpful, please let us know about this issue at
      https://github.com/flutter/flutter/issues.
    ✗ FileSystemException: Exists failed, path = 'C:\Program.plugins' (OS Error: The
      filename, directory name, or volume label syntax is incorrect.
      , errno = 123)
[✓] VS Code (version 1.59.1)
[✓] Connected device (2 available)
```


Mobile App Development

Understanding Flutter, Flutter Framework



Run Flutter Doctor

```
C:\Users\dell>flutter doctor --android-licenses
```

```
C:\Users\dell>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 2.2.3, on Microsoft Windows [Version 10.0.19042.1165],
     locale en-IN)
[✓] Android toolchain - develop for Android devices (Android SDK version 31.0.0)
[✓] Chrome - develop for the web
[✗] Android Studio (the doctor check crashed)
     X Due to an error, the doctor check did not complete. If the error message below is
       not helpful, please let us know about this issue at
       https://github.com/flutter/flutter/issues.
     X FileSystemException: Exists failed, path = 'C:\Program.plugins' (OS Error: The
       filename, directory name, or volume label syntax is incorrect.
       , errno = 123)
[✓] VS Code (version 1.59.1)
[✓] Connected device (2 available)

! Doctor found issues in 1 category.
```




THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOP and Flutter Widgets

Installing Flutter

Dr. Annu Sharma

Department of Computer Applications

A. Understanding Flutter and the Flutter Framework

- Flutter is an open-source UI software development kit (SDK) created by Google.
- It allows developers to create natively compiled applications for mobile (iOS, Android), web, and desktop from a single codebase.

The Flutter Framework includes:

- **Widgets:** The building blocks of a Flutter app. They are immutable and describe part of the user interface. Widgets can be stateless or stateful.
- **Rendering Engine:** Flutter uses Skia to render widgets. It allows drawing to the screen in a performant way. Skia serves as the graphics engine for Google Chrome and ChromeOS, Android, Flutter, and many other products.
- **Platform Channels:** These enable Dart to communicate with native (Android and iOS) code.

Flutter supported IDE's

- Visual Studio Code
 - Lightweight, fast, easy to set up
- Android Studio
- IntelliJ IDEA

A. Configuring Flutter SDK

1. After installing Android Studio, download the Flutter SDK from the [Flutter website] (<https://flutter.dev/docs/get-started/install>).
2. Extract the Flutter SDK to a desired location.
3. Update your system's environment variables to include Flutter's `bin` directory. This allows the Flutter command to be available globally.

Mobile App Development

Understanding Flutter, Flutter Framework



Flutter SDK

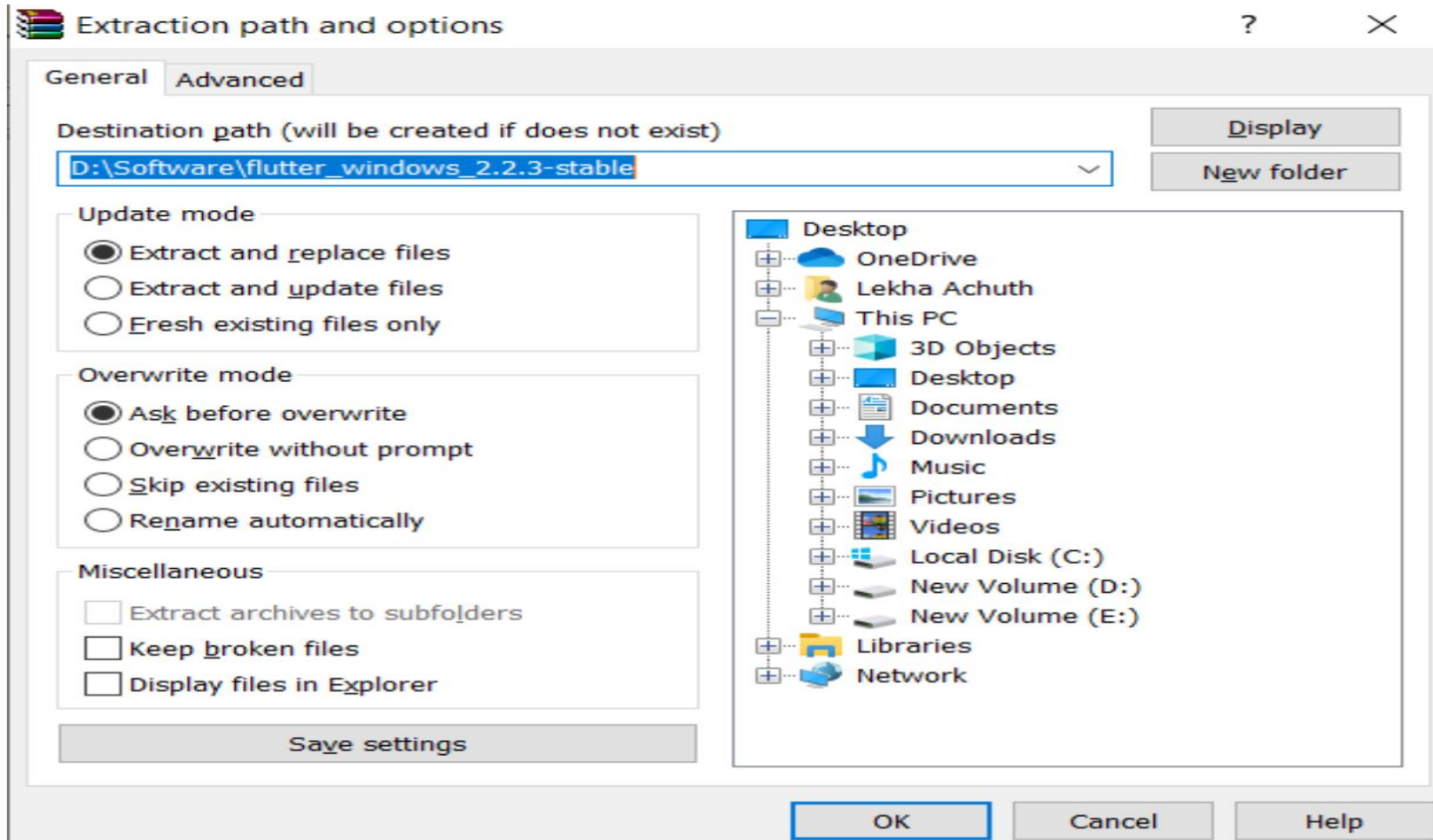
IT includes the Flutter compiler, project creator, device manager, test runner, and tools that diagnose– and even. correct– problems with the Flutter configuration.

A screenshot of the Flutter website's documentation page for installing the Flutter SDK on Windows. The browser address bar shows "flutter.dev/docs/get-started/install/windows". The page has a dark blue header with the Flutter logo and navigation links: Docs, Showcase, Community, and a "Get started" button. A left sidebar lists navigation options like "Get started", "From another platform?", and "Samples & tutorials". The main content area is titled "Get the Flutter SDK" and lists two steps: 1. Download the "flutter_windows_2.2.3-stable.zip" bundle, and 2. Extract the zip file and place the "flutter" folder in a specific directory. A yellow warning box states: "Warning: Do not install Flutter in a directory like C:\Program Files\ that requires elevated privileges." Below the steps, it mentions an alternative method of getting source code from the Flutter repo on GitHub. A right sidebar titled "Contents" lists various setup topics like "System requirements", "Android setup", "Windows setup", and "Web setup".

Mobile App Development

Understanding Flutter, Flutter Framework

Flutter SDK

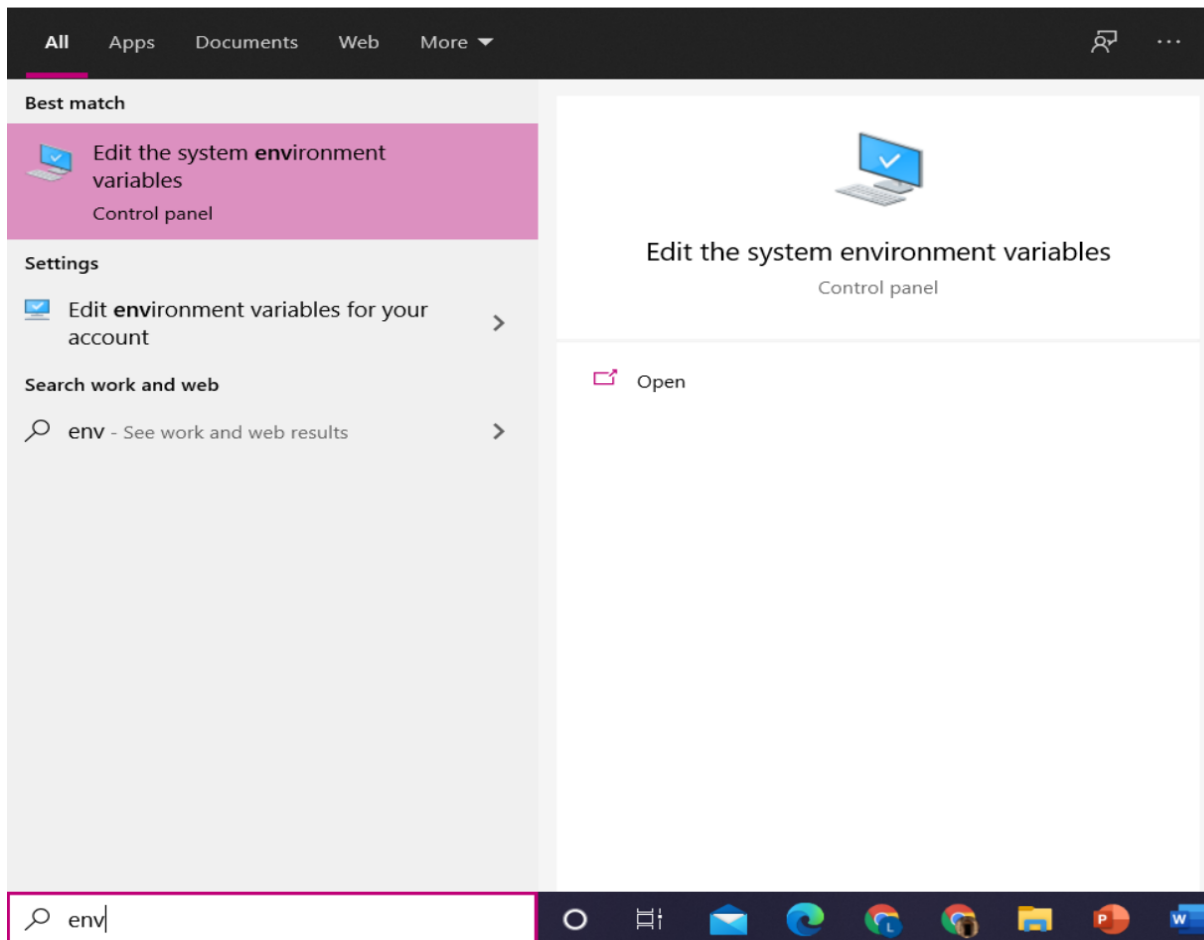


Mobile App Development

Understanding Flutter, Flutter Framework

Edit System variables

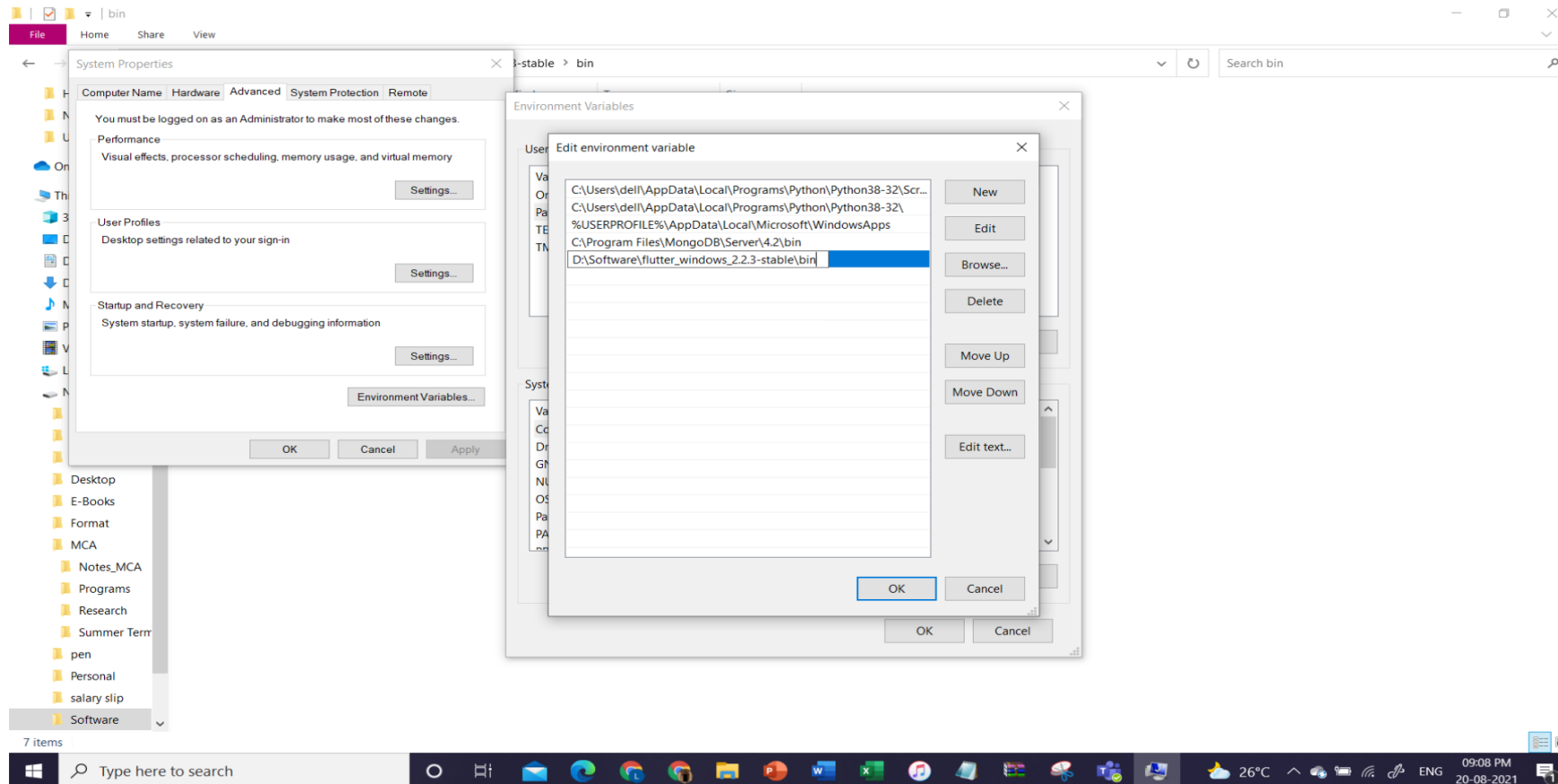
Update the path



Mobile App Development

Understanding Flutter, Flutter Framework

Edit System Environment variables: Update Path



Checking the path setup

```
C:\Users\de11>where flutter dart
D:\Software\flutter_windows_2.2.3-stable\bin\flutter
D:\Software\flutter_windows_2.2.3-stable\bin\flutter.bat
D:\Software\flutter_windows_2.2.3-stable\bin\dart
D:\Software\flutter_windows_2.2.3-stable\bin\dart.bat
```

A. Installing and Configuring Flutter SDK

In addition to configuring the Flutter SDK:

1. Install **Dart SDK** (included with Flutter).
2. Set up an editor like **Android Studio**
3. Install the **Flutter and Dart plugins** from the plugins marketplace.

After installation, check the setup by running the following command in the terminal:

```
flutter doctor
```

This will provide feedback on the state of your Flutter installation and guide you through any missing dependencies.

Mobile App Development

Understanding Flutter, Flutter Framework



Run Flutter Doctor

```
C:\Users\dell>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 2.2.3, on Microsoft Windows [Version 10.0.19042.1165],
    locale en-IN)
[!] Android toolchain - develop for Android devices (Android SDK version 31.0.0)
    ✗ Android licenses not accepted.  To resolve this, run: flutter doctor
      --android-licenses
[✓] Chrome - develop for the web
[?] Android Studio (the doctor check crashed)
    ✗ Due to an error, the doctor check did not complete. If the error message below is
      not helpful, please let us know about this issue at
      https://github.com/flutter/flutter/issues.
    ✗ FileSystemException: Exists failed, path = 'C:\Program.plugins' (OS Error: The
      filename, directory name, or volume label syntax is incorrect.
      , errno = 123)
[✓] VS Code (version 1.59.1)
[✓] Connected device (2 available)
```


Mobile App Development

Understanding Flutter, Flutter Framework



Run Flutter Doctor

```
C:\Users\dell>flutter doctor --android-licenses
```

```
C:\Users\dell>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 2.2.3, on Microsoft Windows [Version 10.0.19042.1165],
    locale en-IN)
[✓] Android toolchain - develop for Android devices (Android SDK version 31.0.0)
[✓] Chrome - develop for the web
[✗] Android Studio (the doctor check crashed)
    X Due to an error, the doctor check did not complete. If the error message below is
      not helpful, please let us know about this issue at
      https://github.com/flutter/flutter/issues.
    X FileSystemException: Exists failed, path = 'C:\Program.plugins' (OS Error: The
      filename, directory name, or volume label syntax is incorrect.
      , errno = 123)
[✓] VS Code (version 1.59.1)
[✓] Connected device (2 available)

! Doctor found issues in 1 category.
```




THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOP and Flutter Widgets

Installing Android Studio

Dr. Annu Sharma

Department of Computer Applications

Mobile App Development

Installing Android Studio



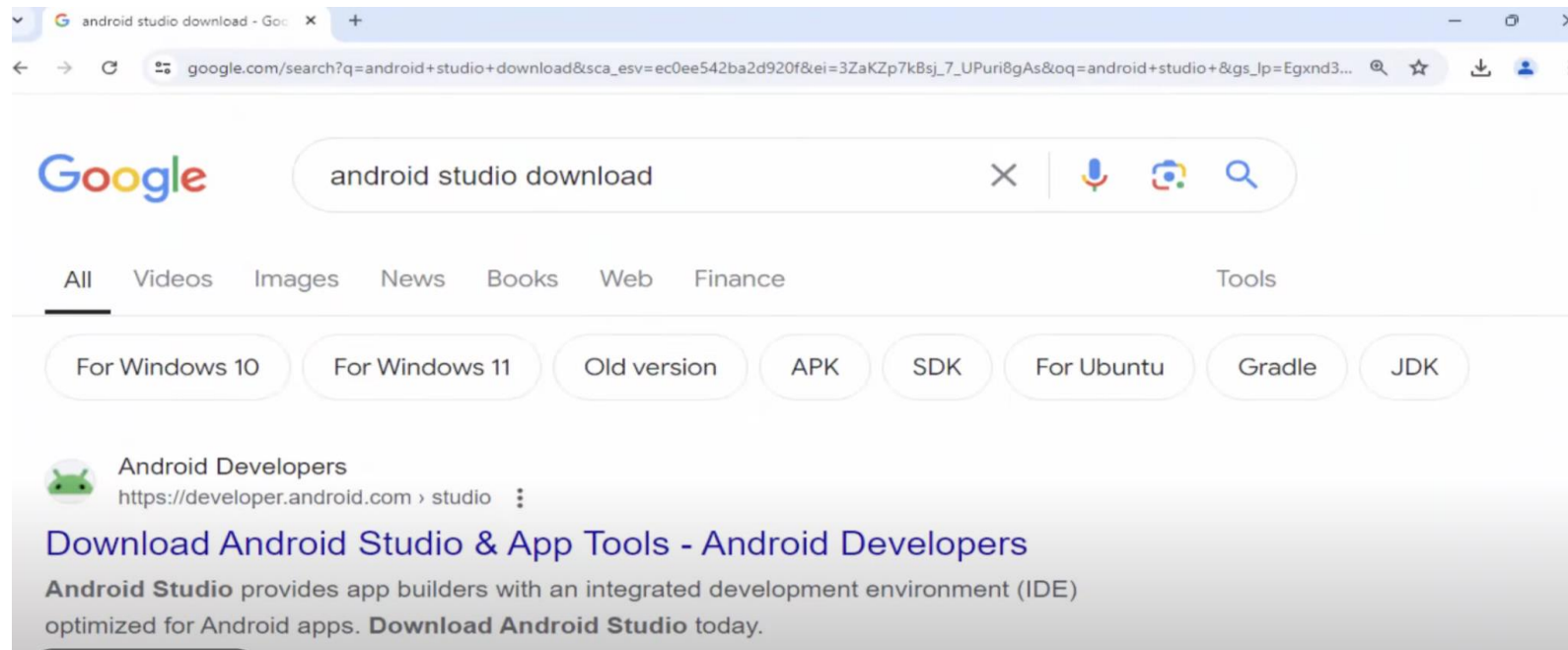
To develop Flutter apps, you need to install **Android Studio**, which provides a complete development environment:

1. Download and install Android Studio from the [official website](<https://developer.android.com/studio>).
2. Follow the installation prompts.
3. During setup, select the option to install the Android SDK, Emulator, and Android Virtual Device (AVD).

Mobile App Development

Installing Android Studio

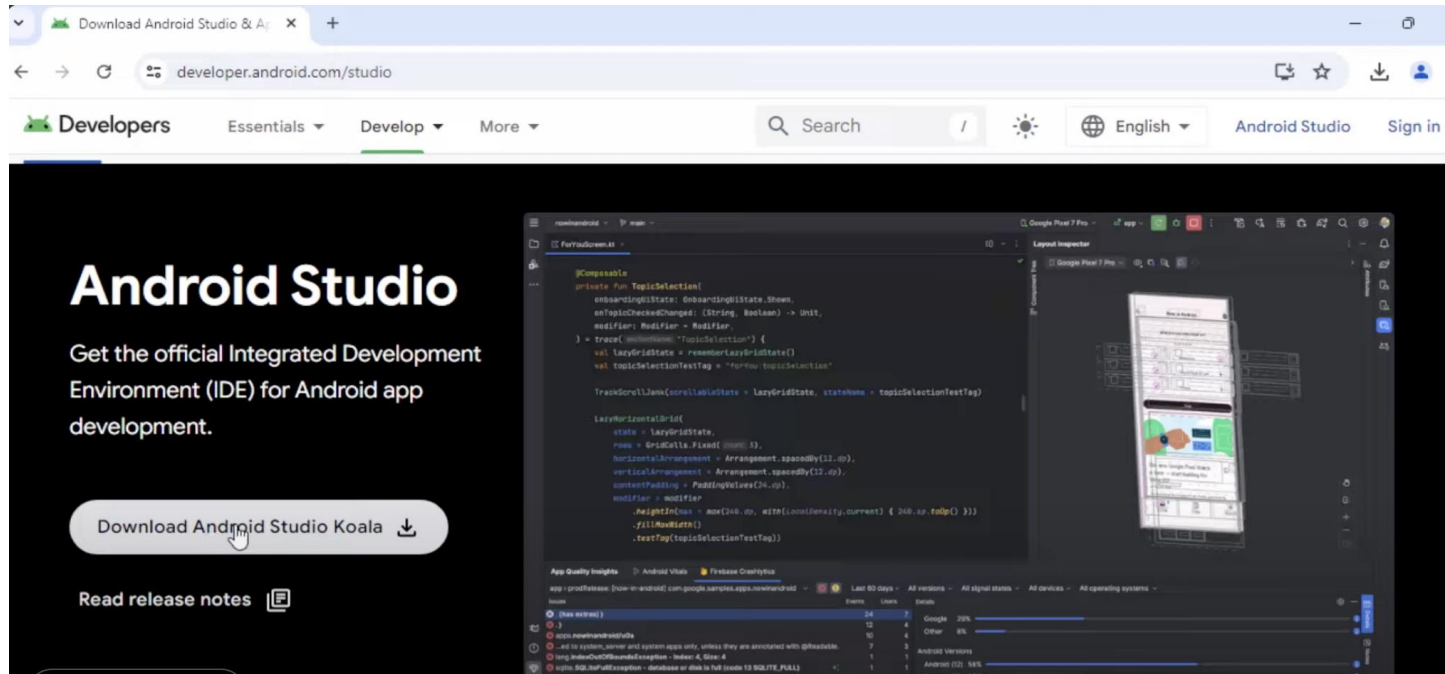
Step1 : Click on the Link Andriod Developer



Mobile App Development

Installing Android Studio

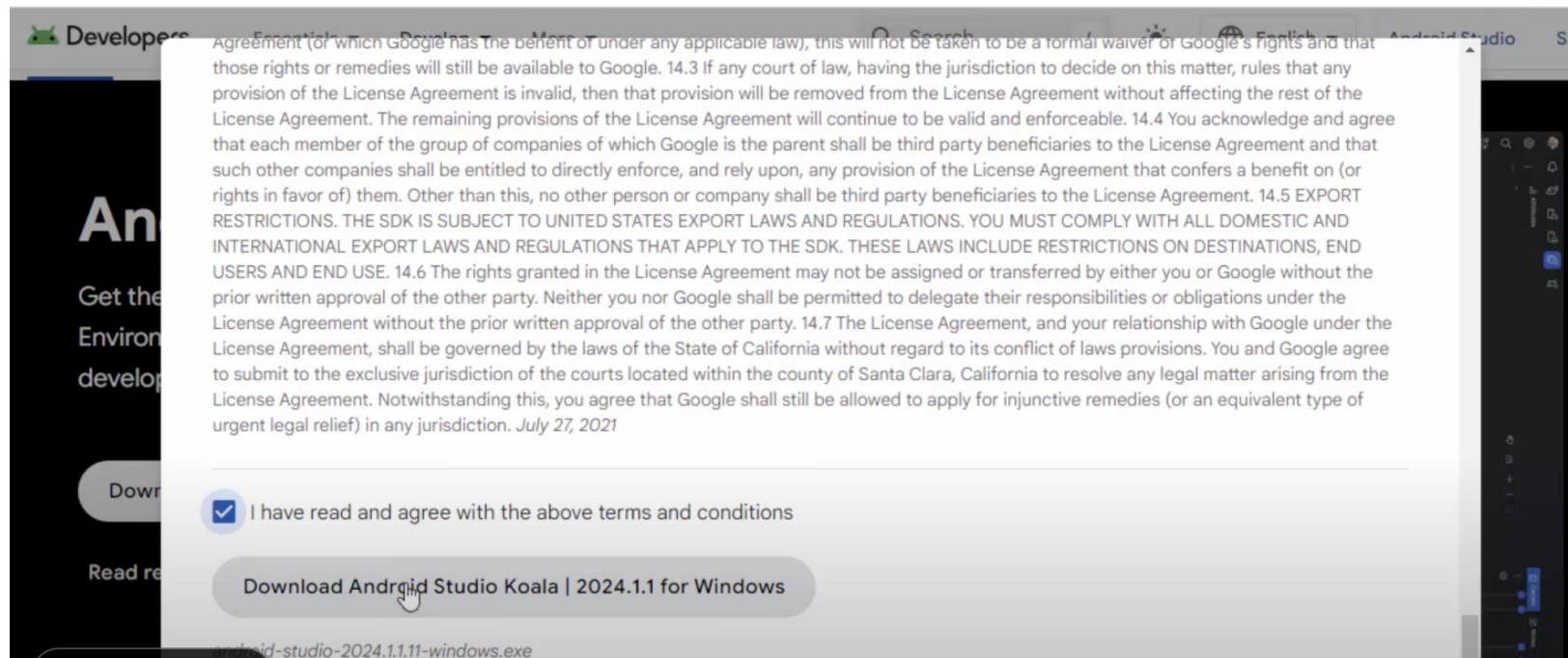
Step2 : Click on the Download Android Studio



Mobile App Development

Installing Android Studio

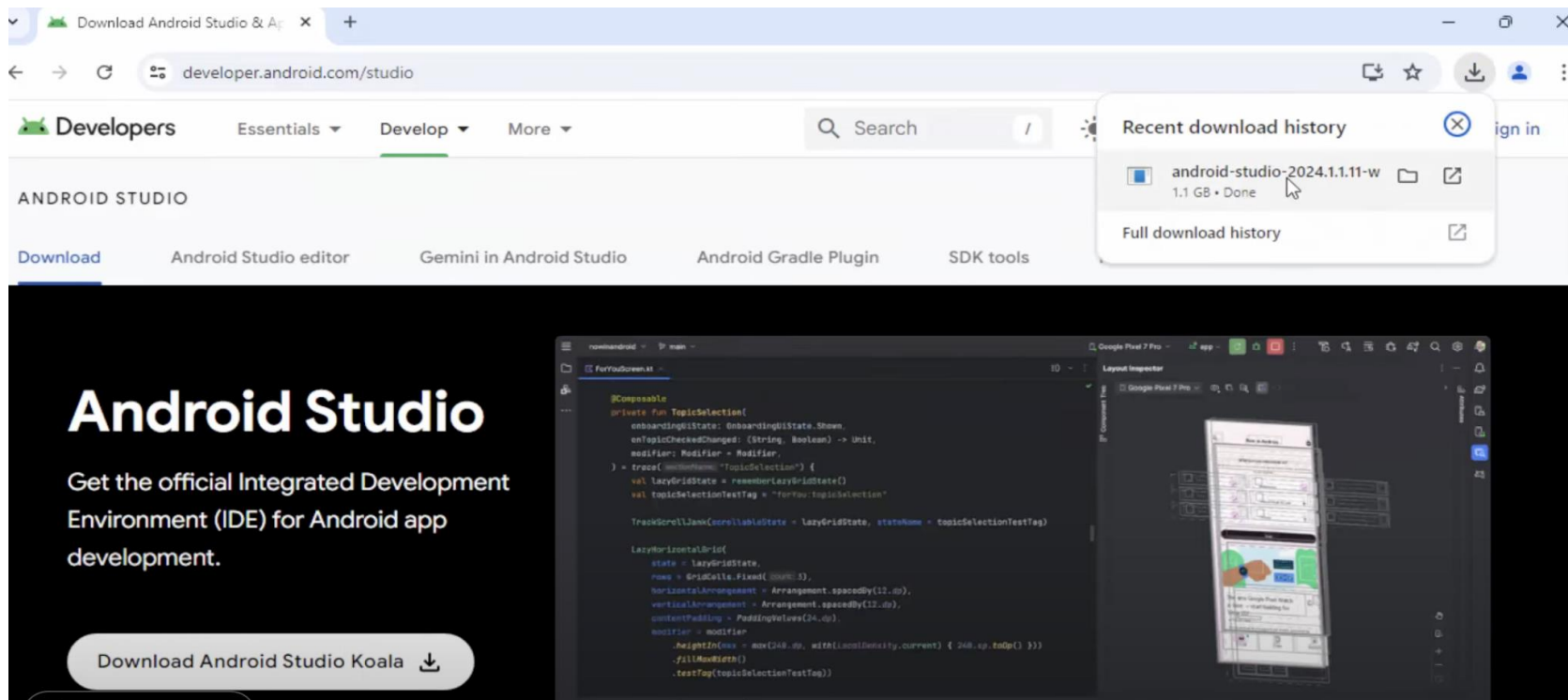
Step3 : Click on the Agree and download



Mobile App Development

Installing Android Studio

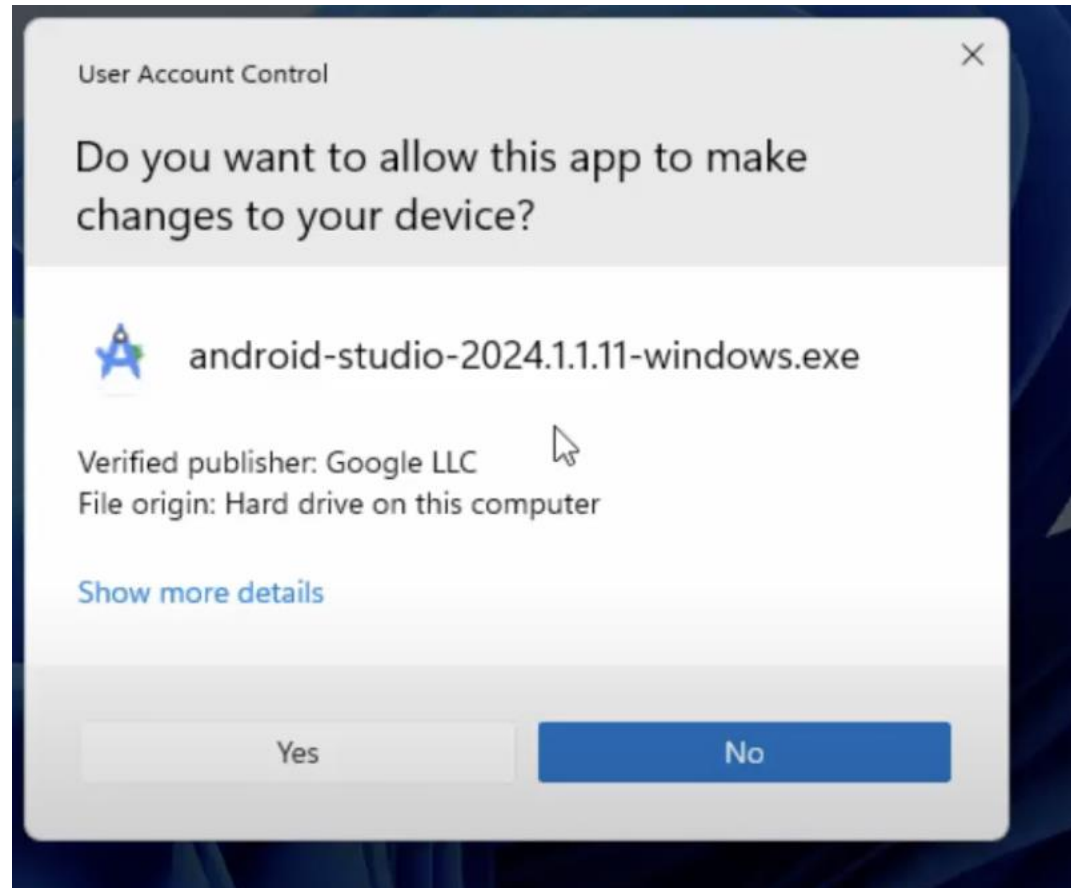
Step4 : Click on .exe file will get downloaded



Mobile App Development

Installing Android Studio

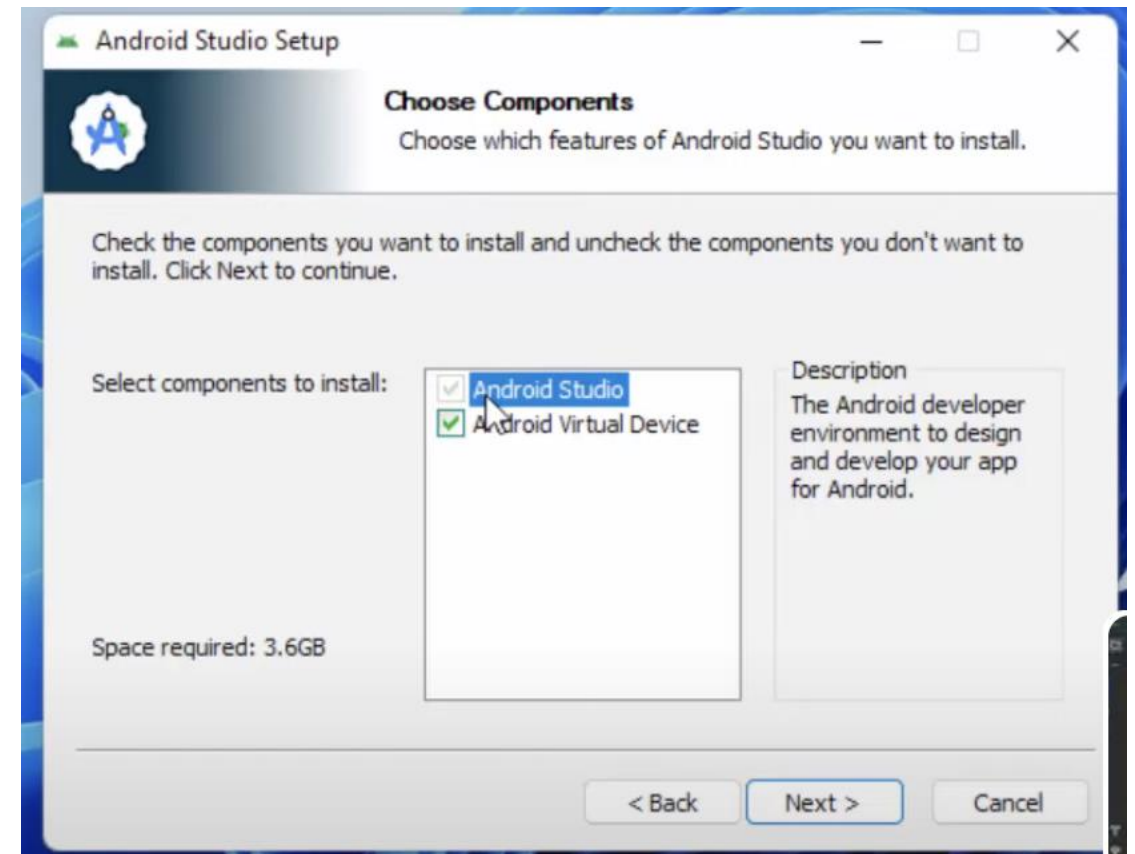
Step5 : Click on Yes



Mobile App Development

Installing Android Studio

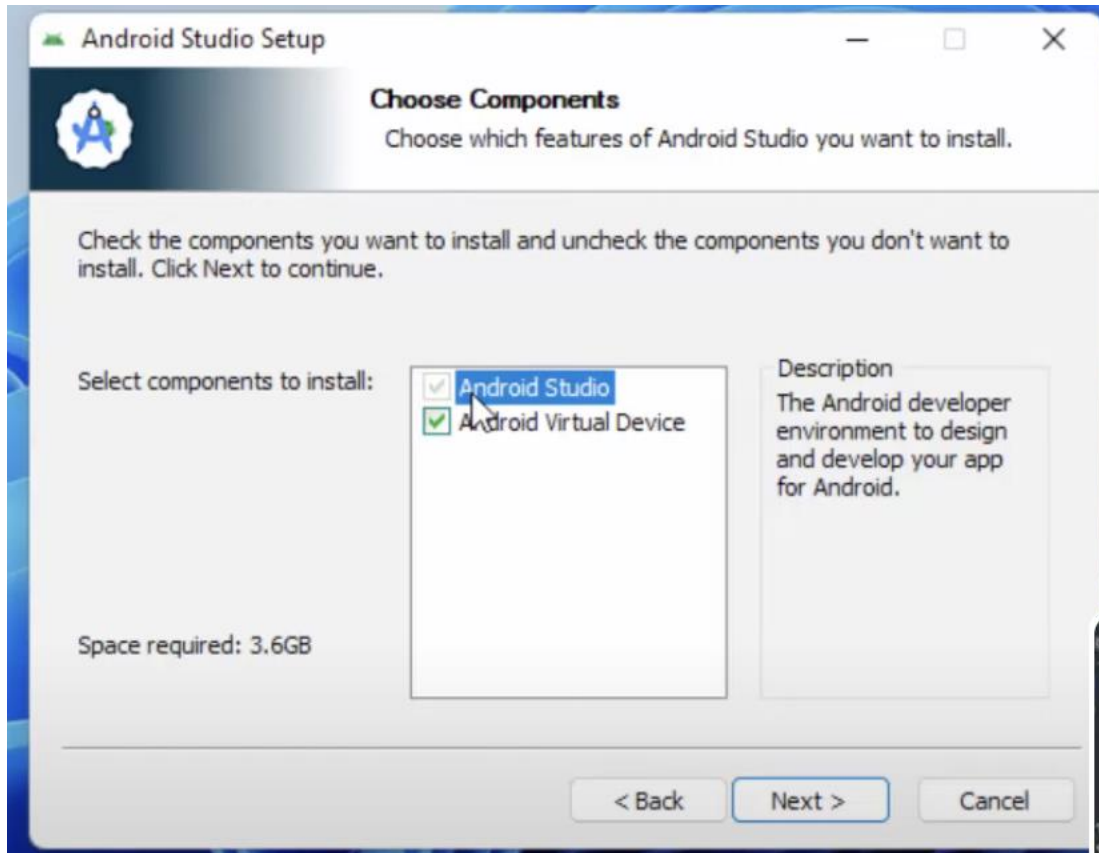
Click on Next



Mobile App Development

Installing Android Studio

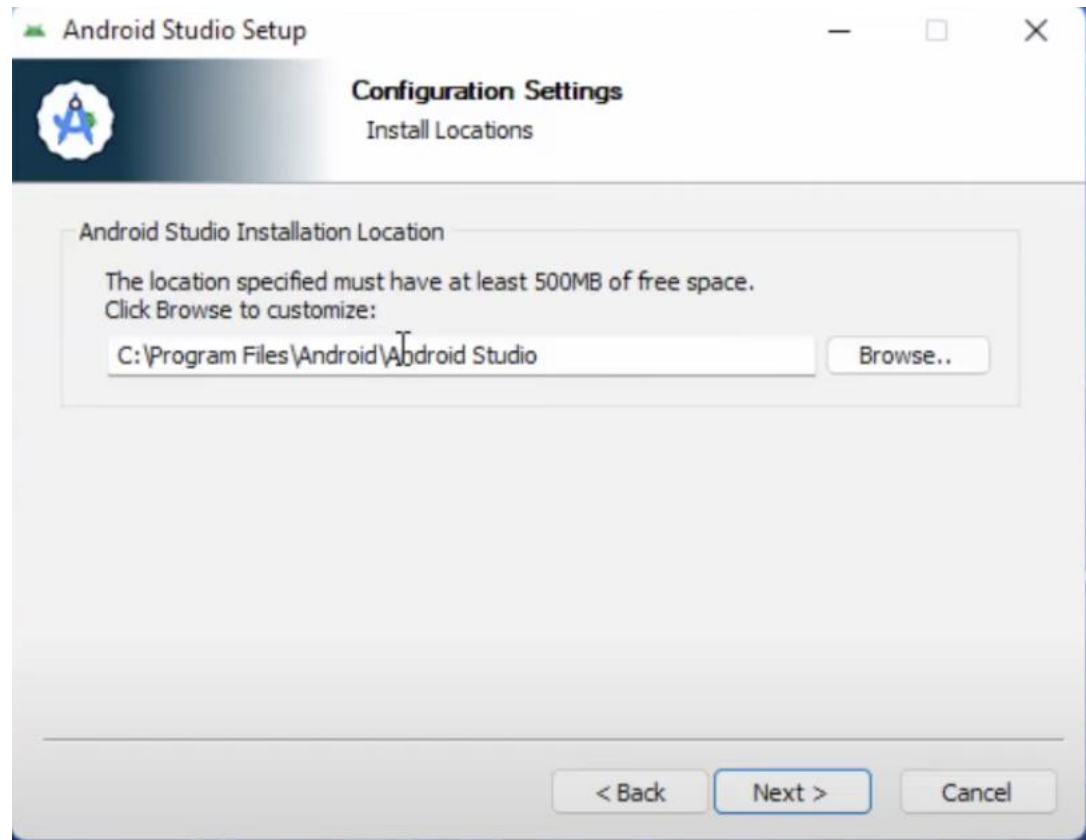
Click on Next



Mobile App Development

Installing Android Studio

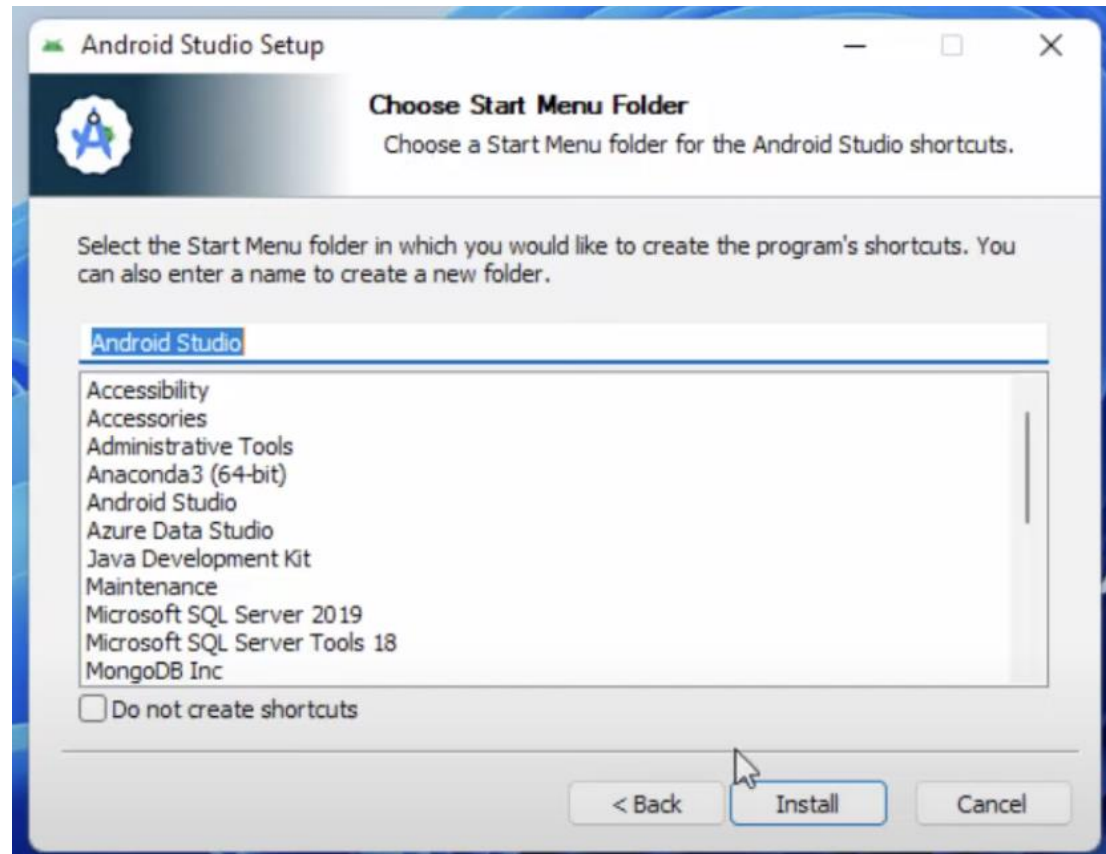
Step6 : Location where Android Studio will be Installed Click on Next



Mobile App Development

Installing Android Studio

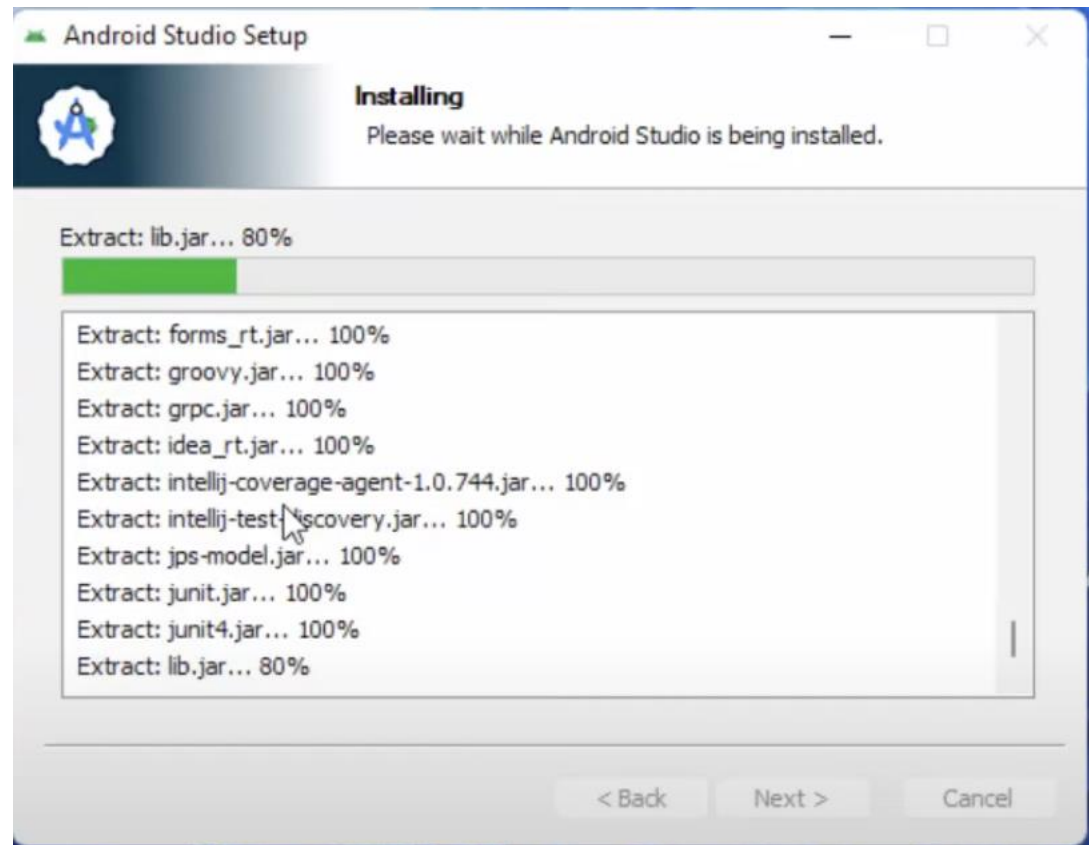
Step7 : Click on Install



Mobile App Development

Installing Android Studio

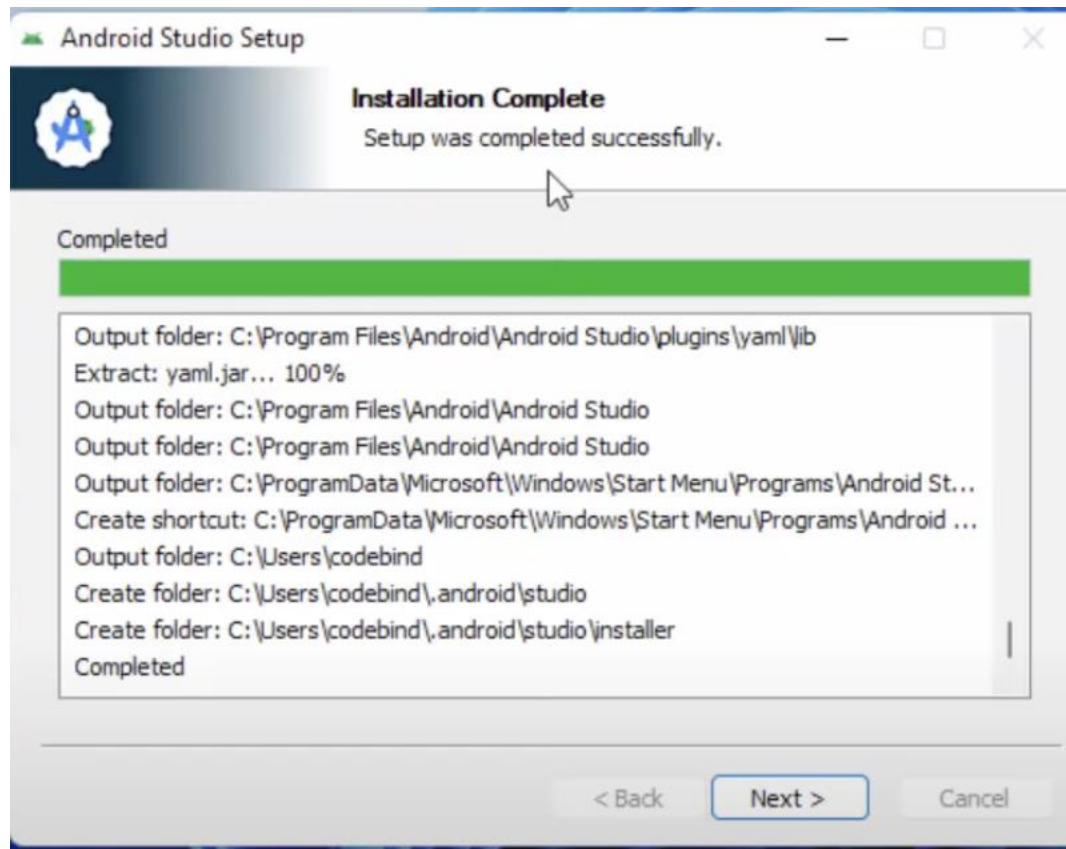
Step8 : Click on Show details will show Installation Progress



Mobile App Development

Installing Android Studio

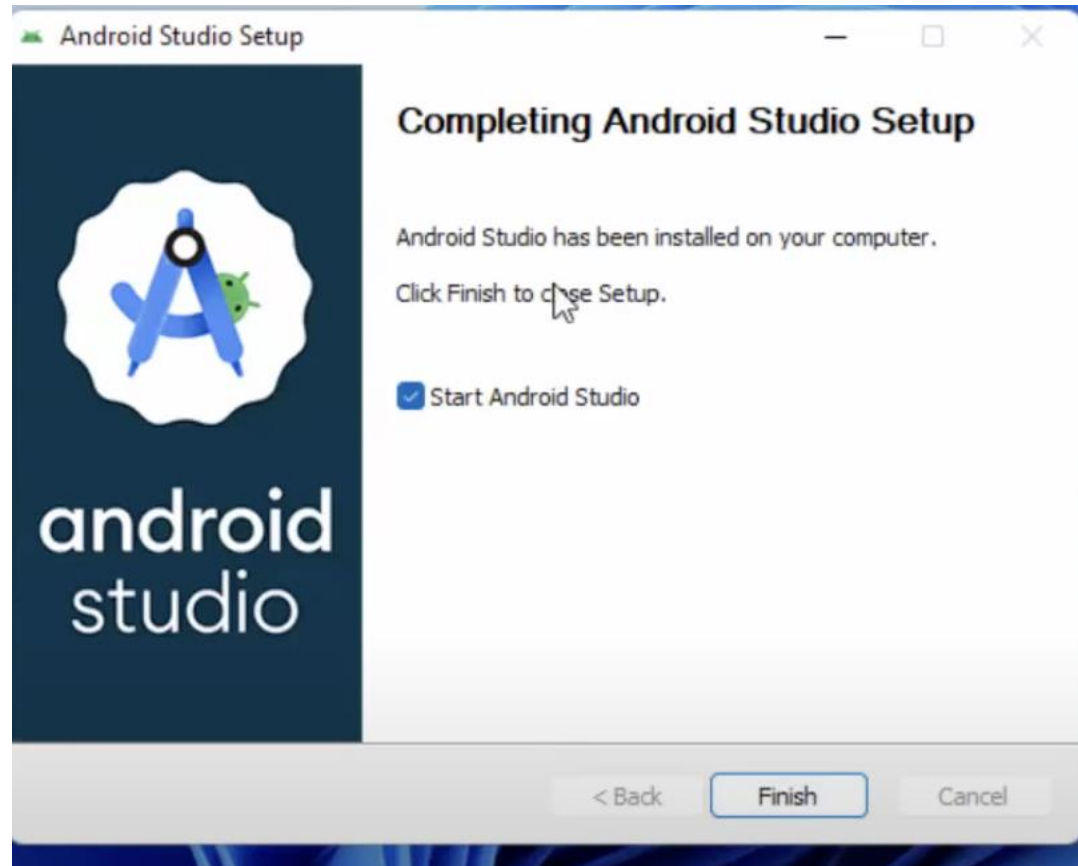
Step8 : Wait till it shows Installation Complete and then Click Next



Mobile App Development

Installing Android Studio

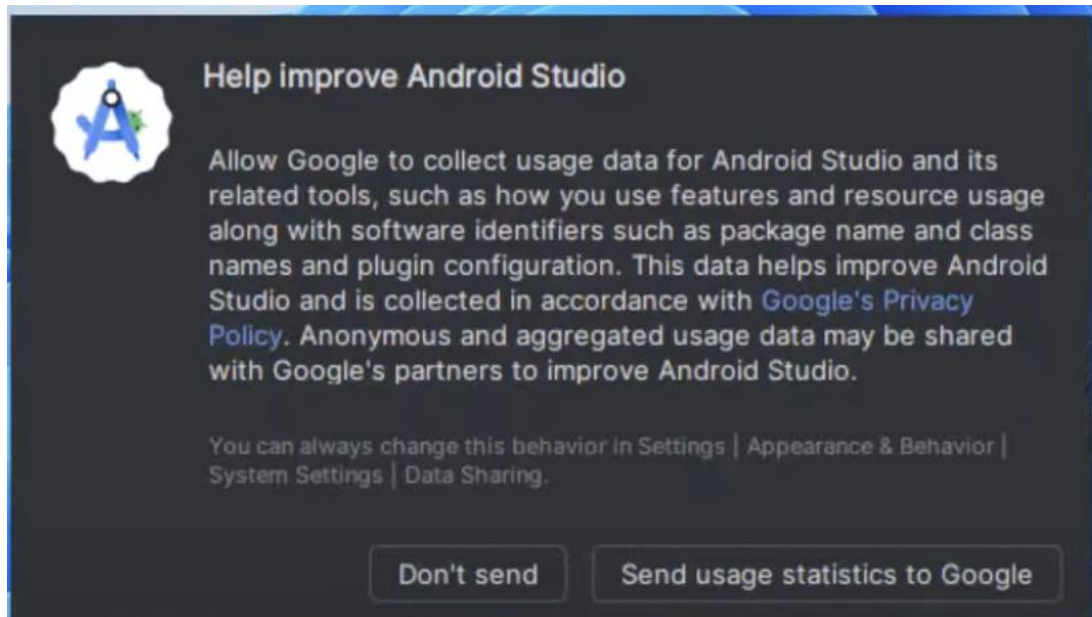
Step9 : Click on Finish



Mobile App Development

Installing Android Studio

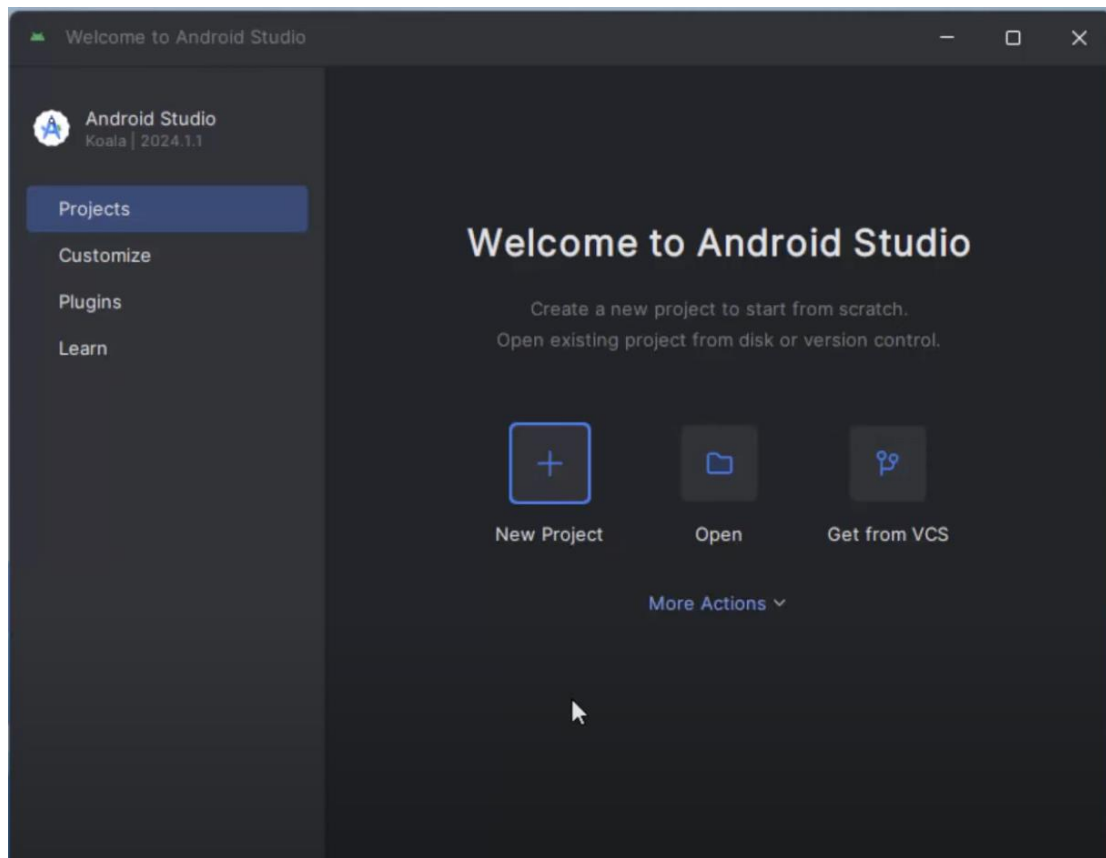
Step10 : First Time when you Launch Android Studio , Click on Don't Send



Mobile App Development

Installing Android Studio

Step11: Android Studio is being Installed on your OS Click on New Project





THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOP and Flutter Widgets

Setup an Android Virtual Device

Dr. Annu Sharma
Department of Computer Applications

Mobile App Development

Setup an Android Virtual Device



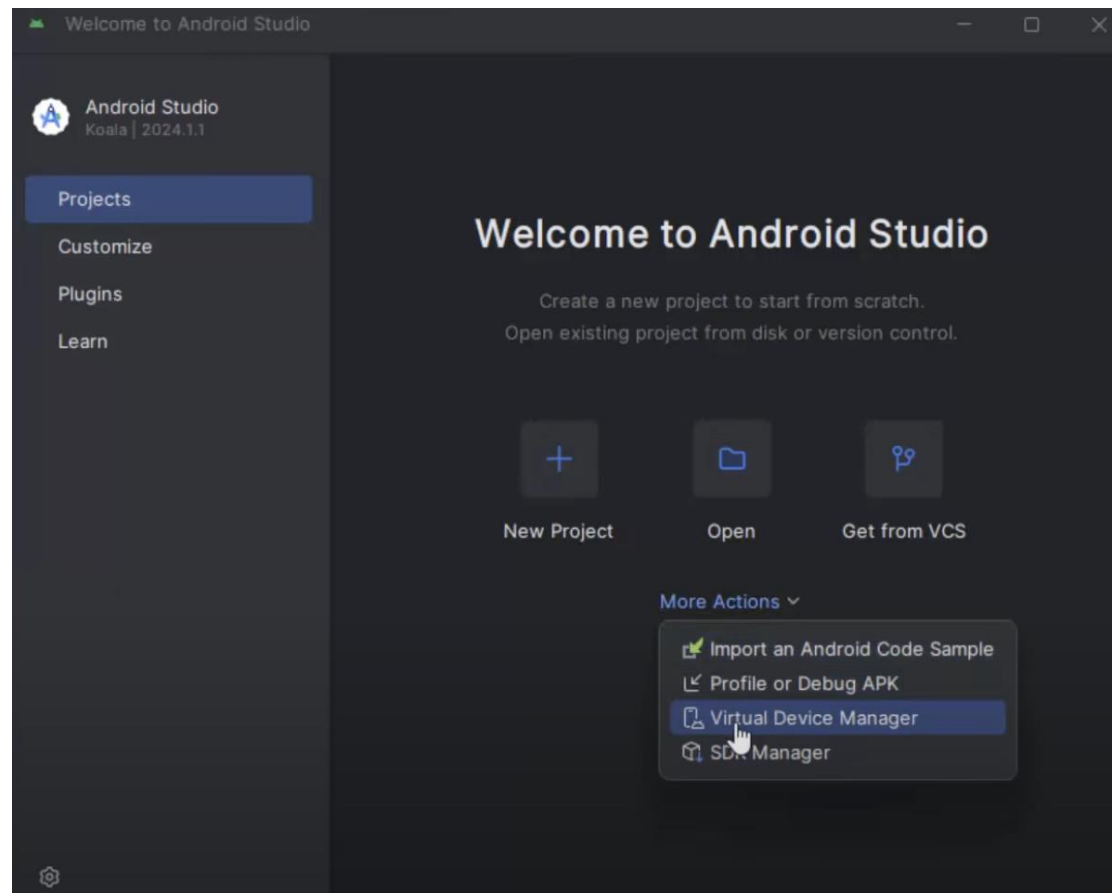
Setting Up an Android Virtual Device (AVD)

- An **Android Virtual Device (AVD)** is required to emulate Android apps without a physical device. To set one up:
 - Open **Android Studio**.
 - Go to **Tools > AVD Manager**.
 - Click **Create Virtual Device** and follow the prompts to choose a device profile (like Pixel) and an Android system image.
 - After setting up, you can launch the emulator to simulate an Android phone.

Mobile App Development

Setup an Android Virtual Device

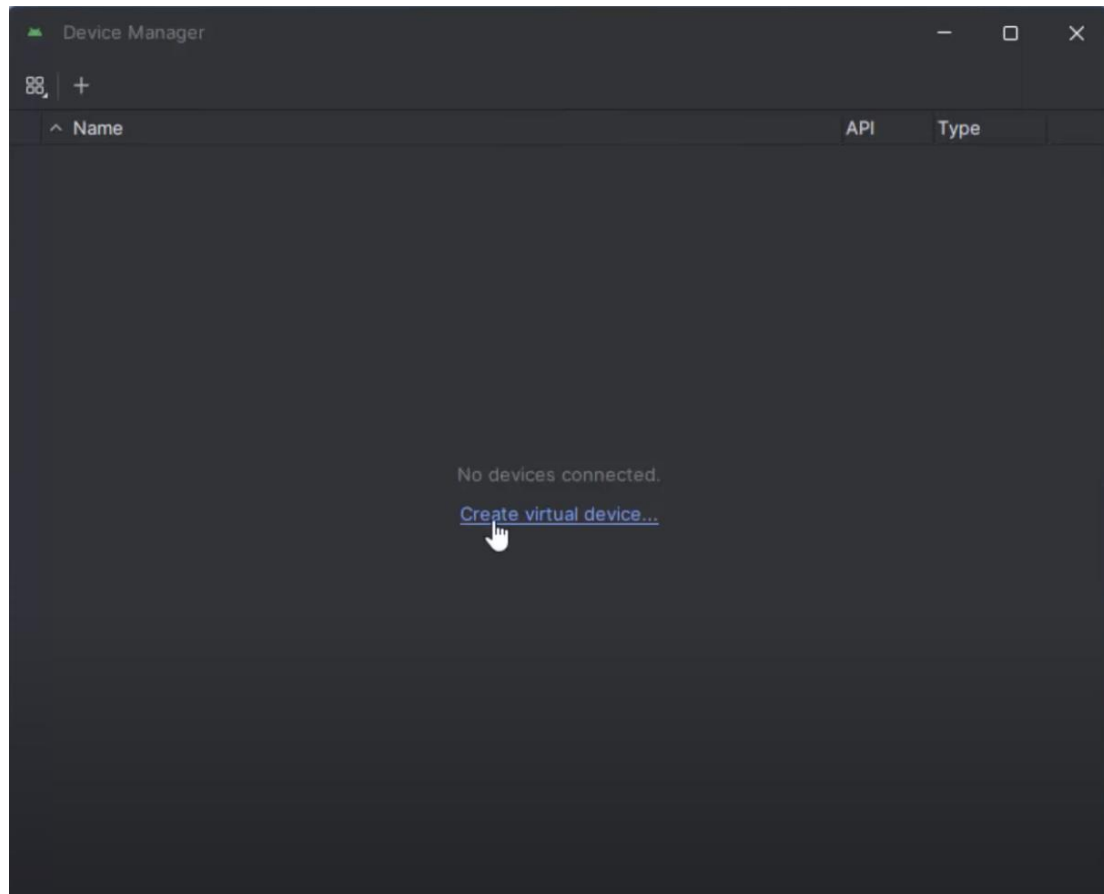
Step-1 Click on Virtual Device Manager



Mobile App Development

Setup an Android Virtual Device

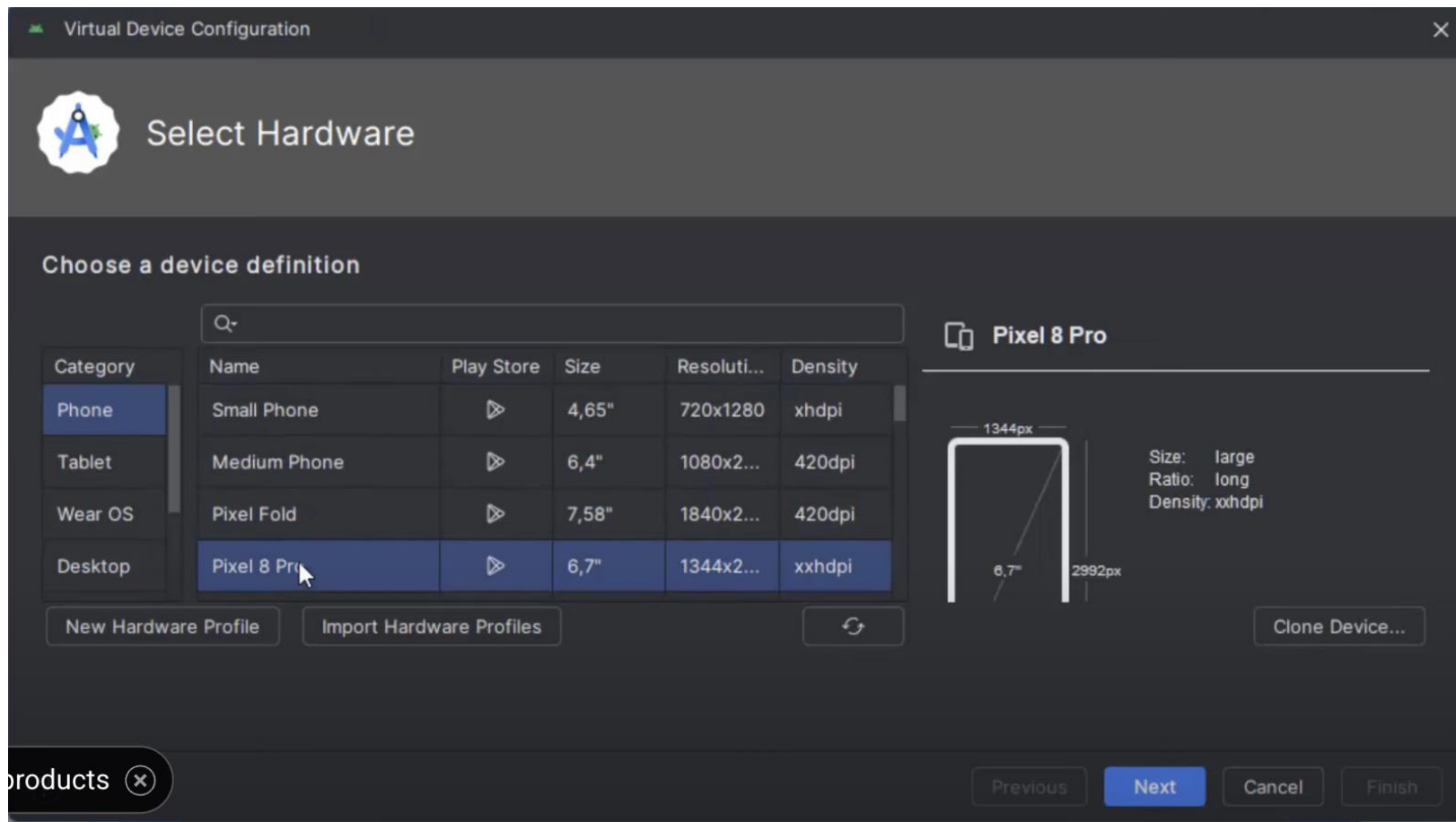
Step-2 Click on Create Virtual Device



Mobile App Development

Setup an Android Virtual Device

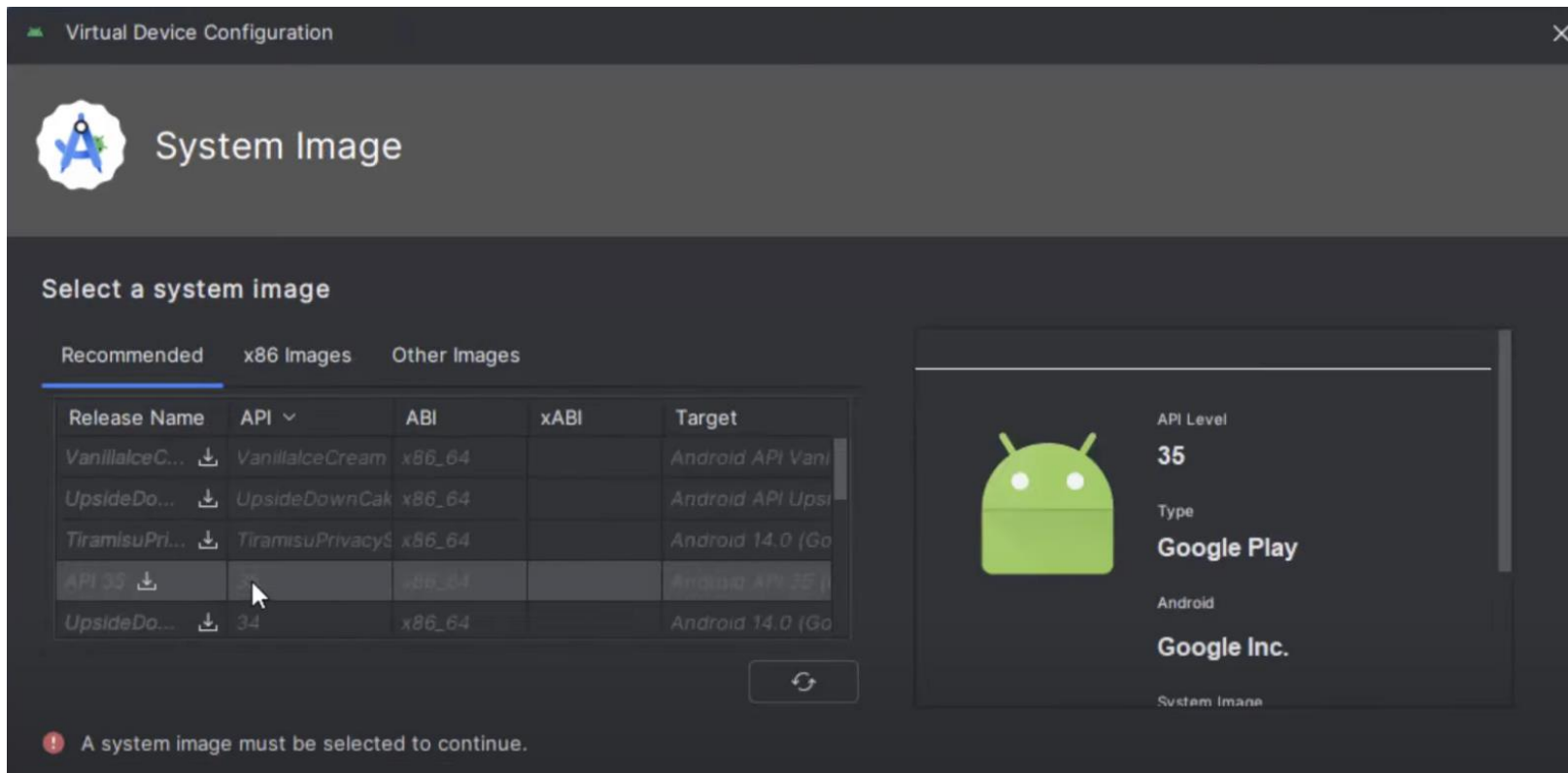
Step-3 : You can select any of the category .Select Phone and pixel 8 Pro and Click on Next



Mobile App Development

Setup an Android Virtual Device

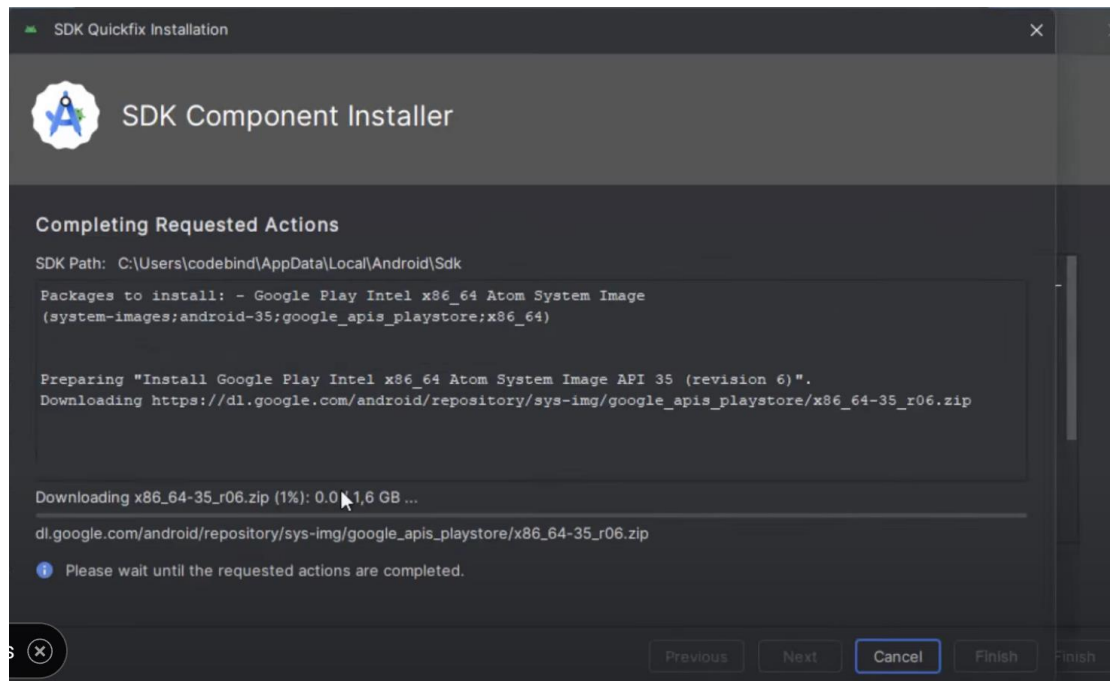
Step-4 : Now select System Image, Let say select API 35 click Next



Mobile App Development

Setup an Android Virtual Device

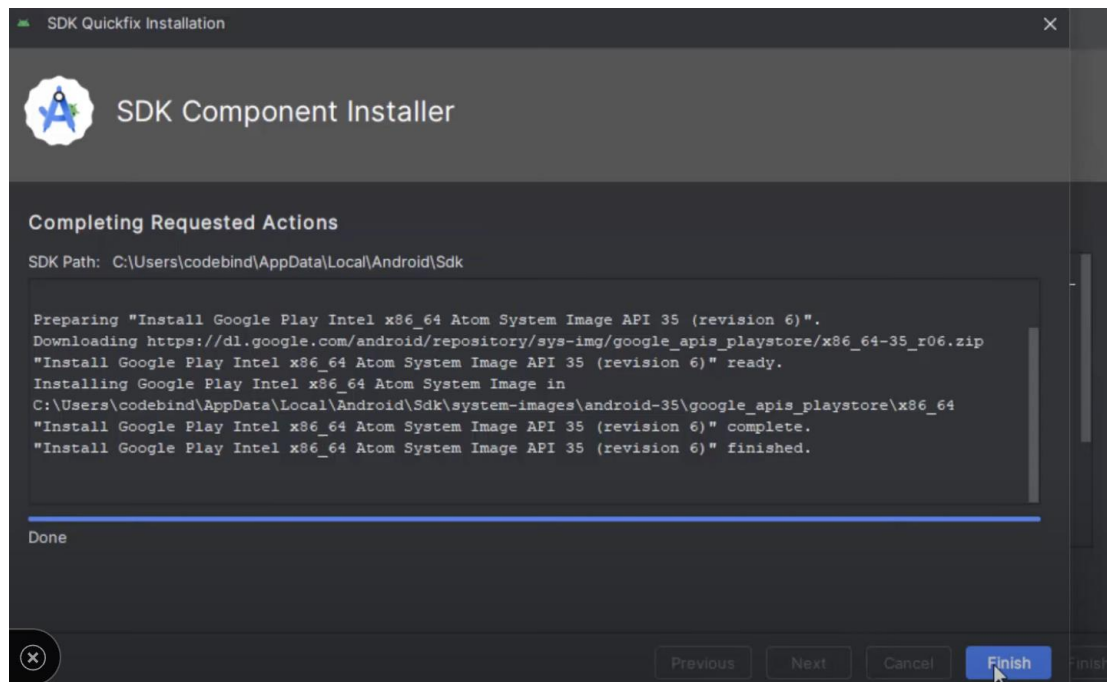
Step-5 : Click Download API 35 . You can see SDK component Installed



Mobile App Development

Setup an Android Virtual Device

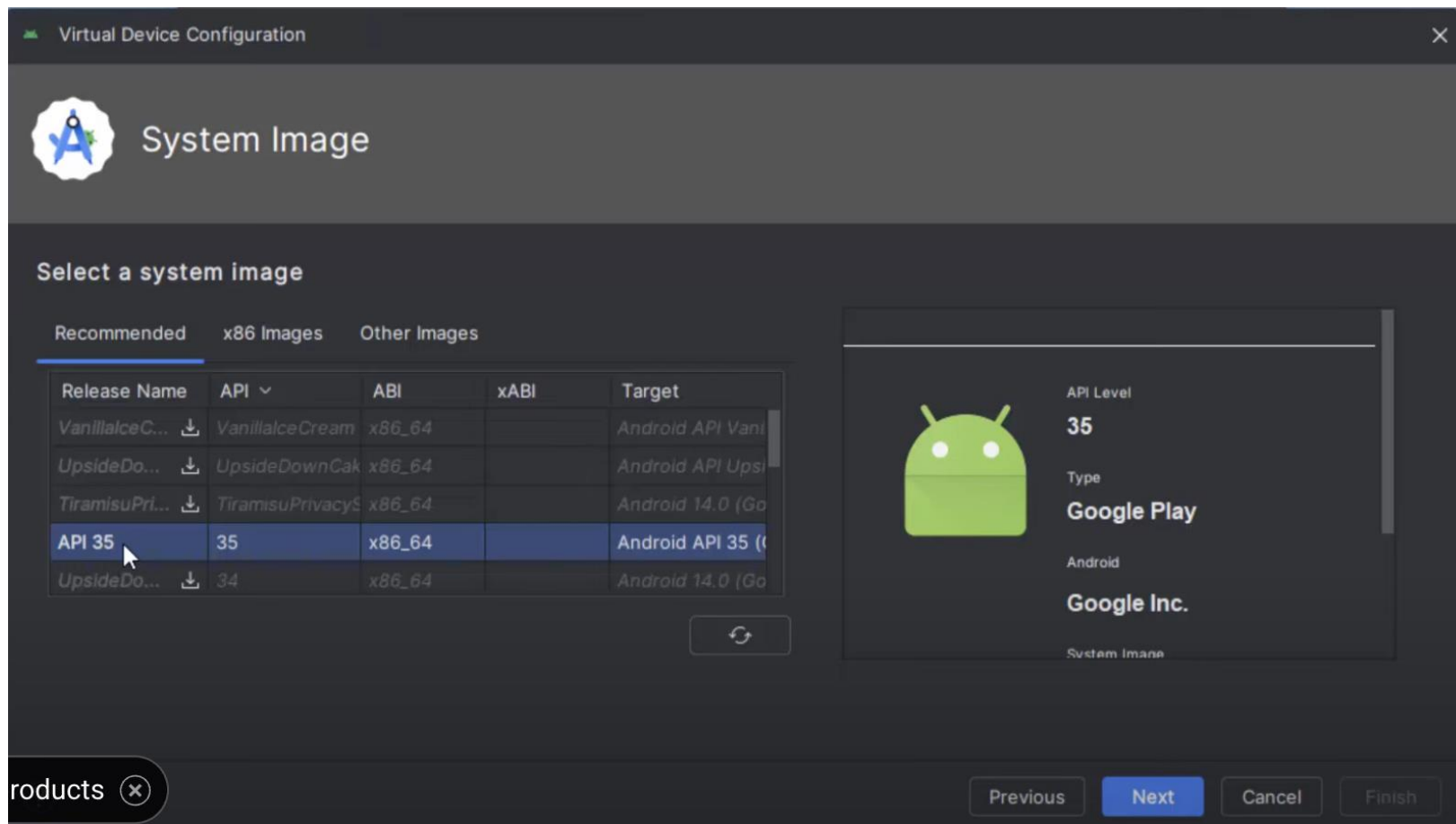
Step-6 : You can see SDK component Installed when it is complete click on Finish Button



Mobile App Development

Setup an Android Virtual Device

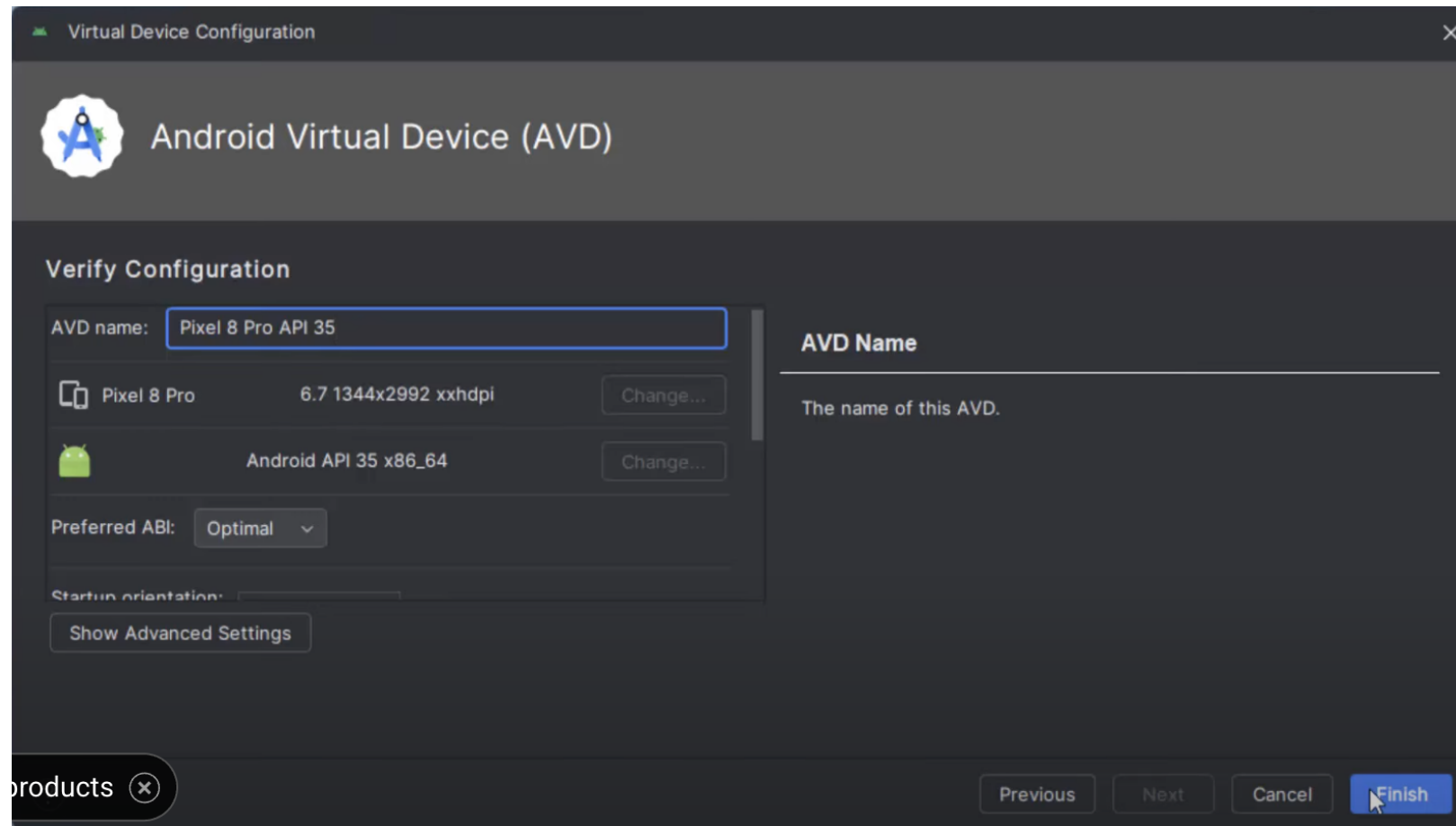
Step-7 : Next Select API 35



Mobile App Development

Setup an Android Virtual Device

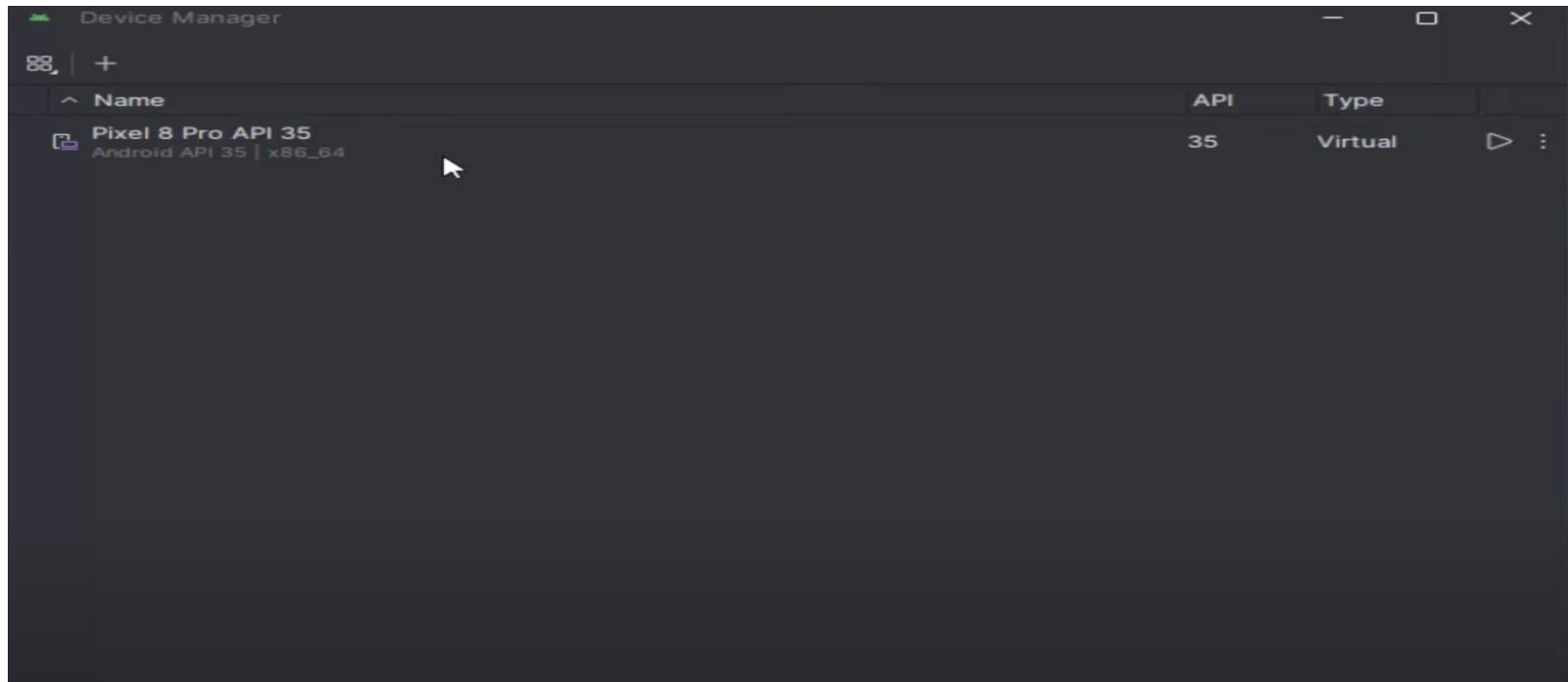
Step-8 : Check AVD Name leave everything as default and Click Finish



Mobile App Development

Setup an Android Virtual Device

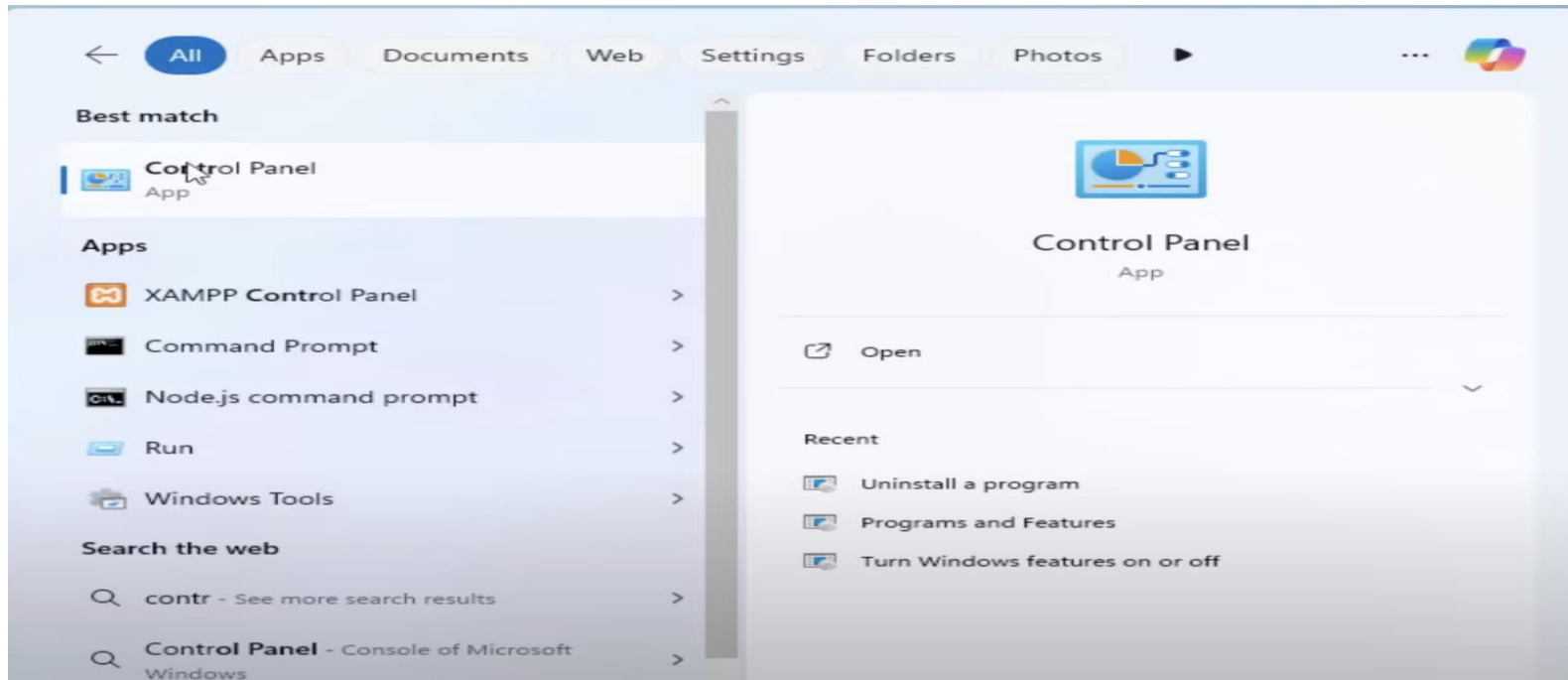
Step-9 : Once you Click Ok your virtual Device is Ready



Mobile App Development

Setup an Android Virtual Device

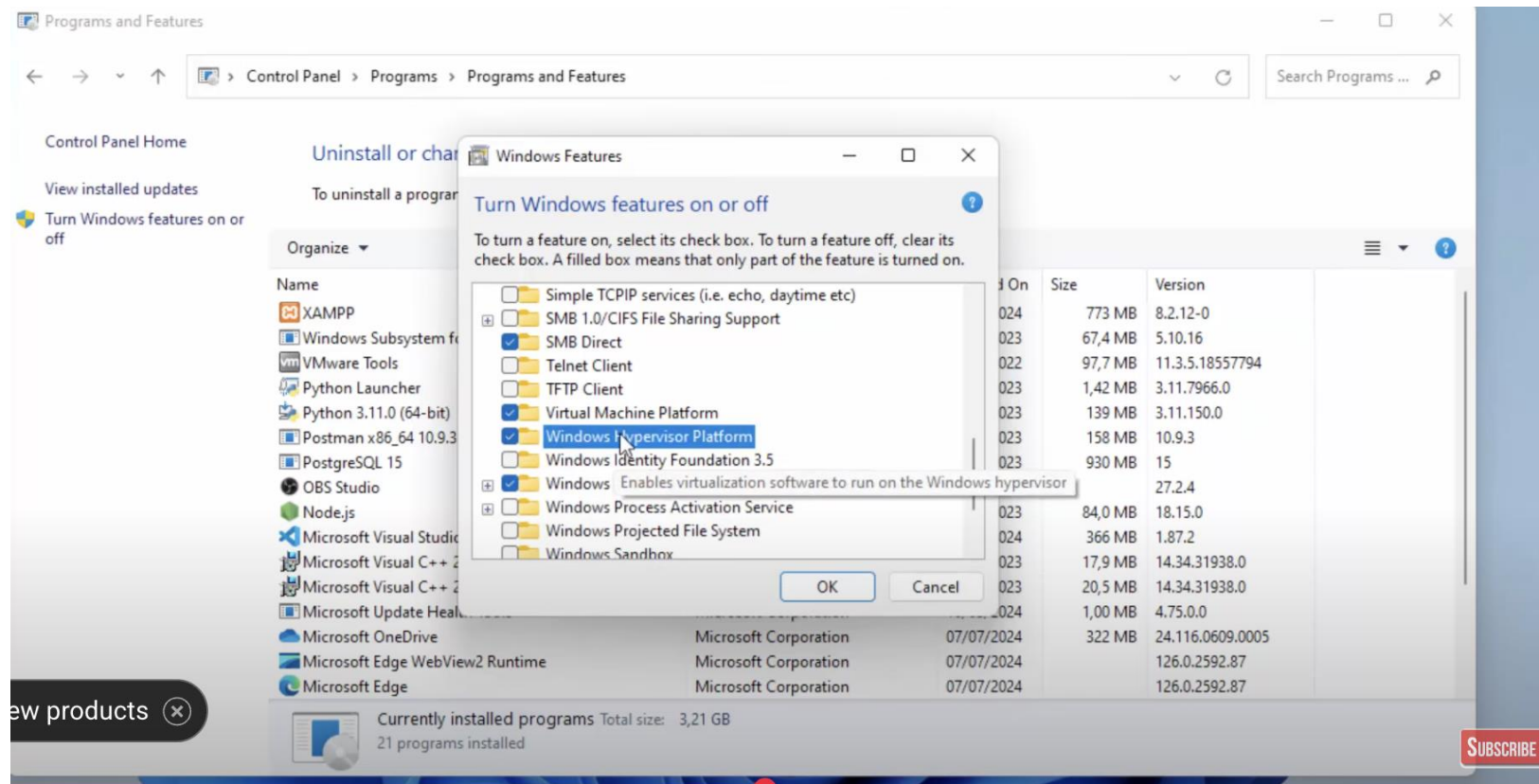
Step-10 : Close Andriod Studio and search for control Panel and click on it



Mobile App Development

Setup an Android Virtual Device

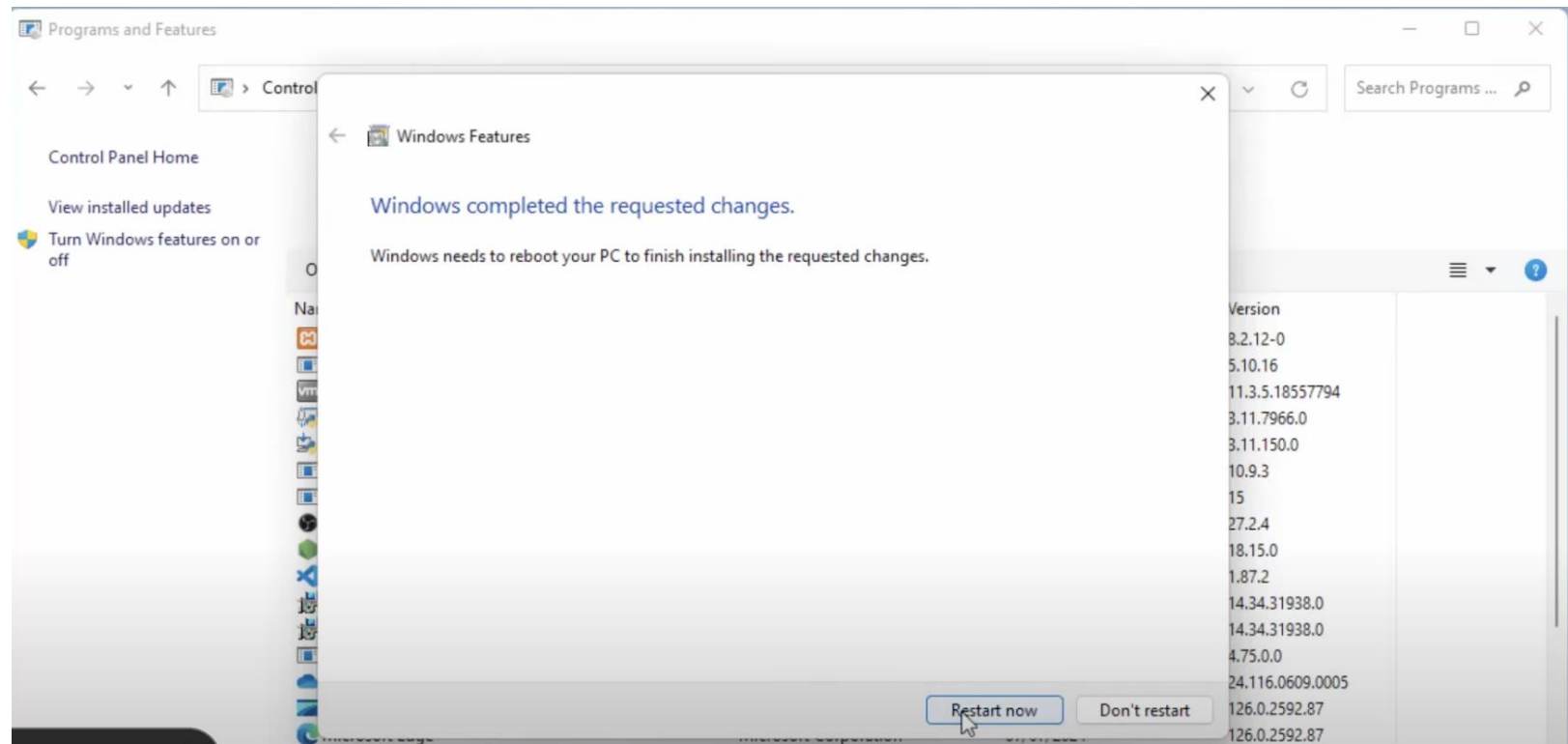
Step-10 : To Enable your Virtual Device go to Control Panel→Programs -> Program features then click on Windows Hypervisor Platform and also select Hyper-V Click ok



Mobile App Development

Setup an Android Virtual Device

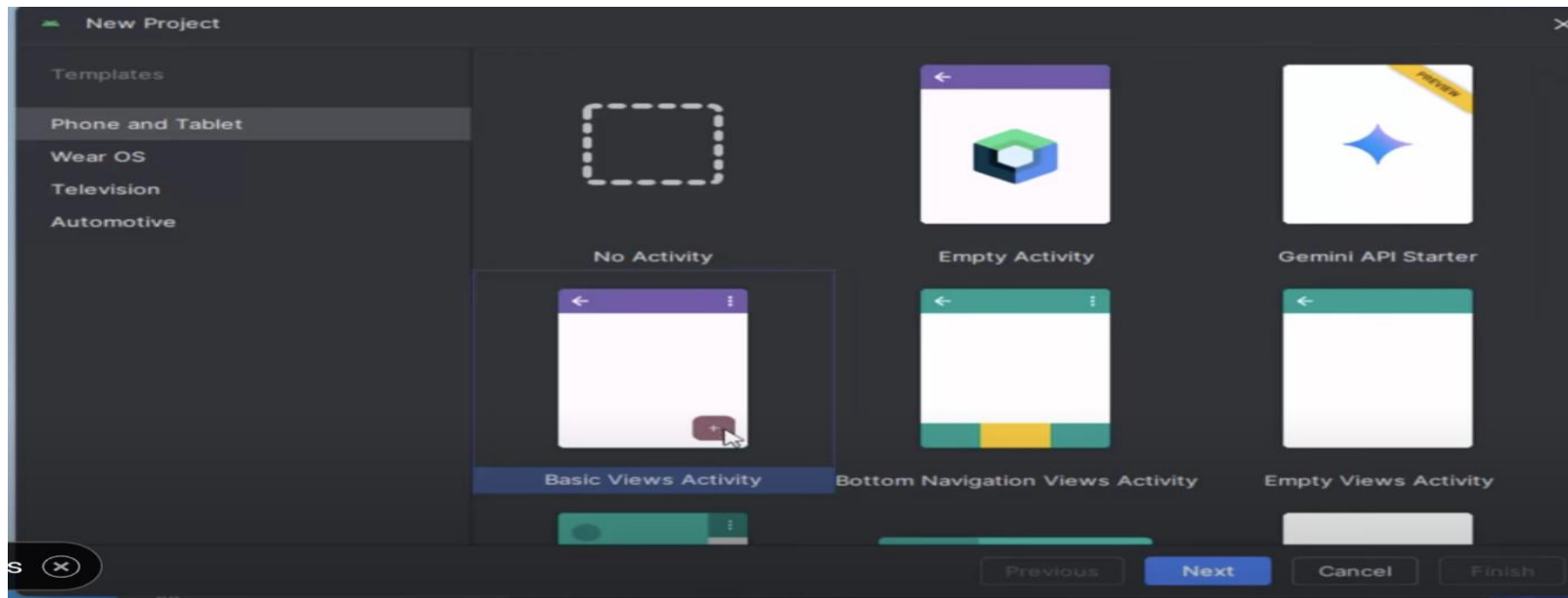
Step-10 : Once its complete it will show you Windows completed the requested Changes then Click on Restart



Mobile App Development

Setup an Android Virtual Device

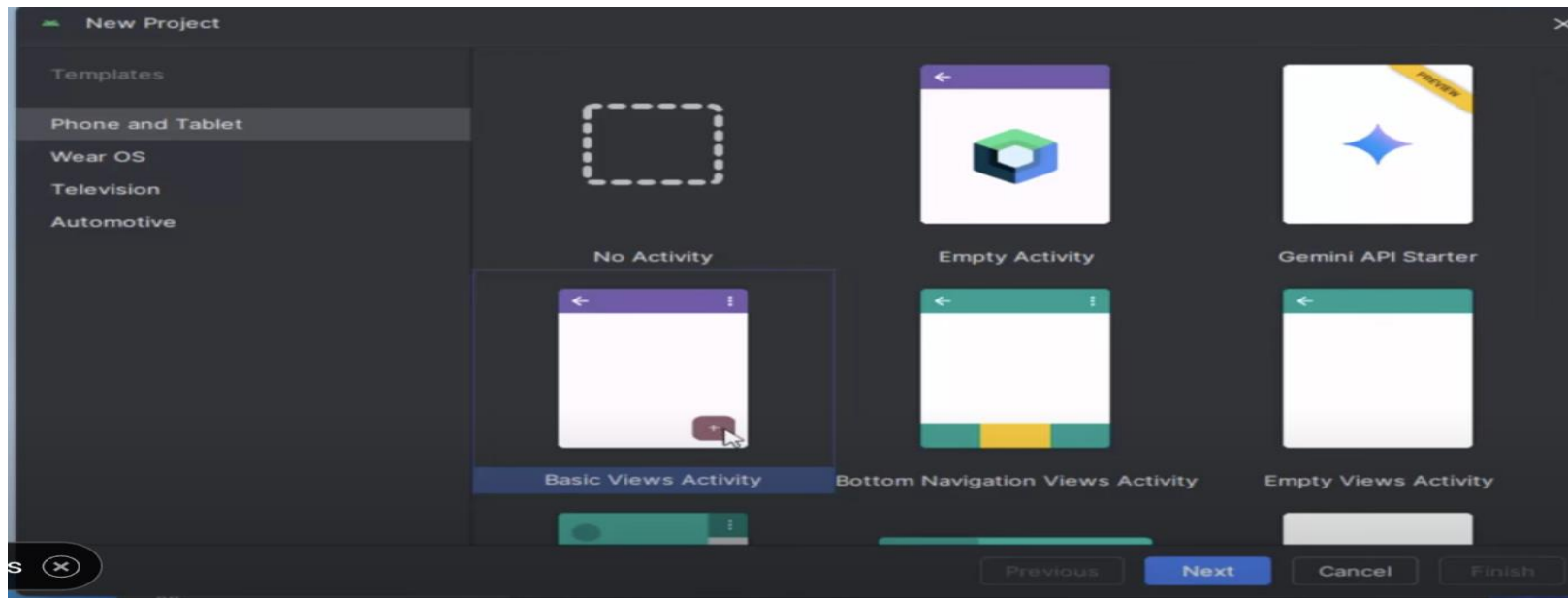
Step-11 : Now go to Android Studio and select New Project with basic view activity



Mobile App Development

Setup an Android Virtual Device

Step-11 : Now go to Andriod Studio and select New Projct with basic view activity for creating new application





THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science



MOBILE APP DEVELOPMENT

(UQ23CA243AA1)

Dr. Annu Sharma
Associate Professor

Department of Computer Applications

MOBILE APP DEVELOPMENT

OOP and Flutter Widgets

Creating Simple Flutter App

Dr. Annu Sharma

Department of Computer Applications

Mobile App Development

Creating Simple Flutter App



Running Your Flutter App

You can run your Flutter app in several ways:

- Via **Android Studio**: Press the play button or use `Shift + F10`.

Understanding a Simple Flutter App Structure

A basic Flutter program typically involves a few essential components. Below is the basic structure of a Flutter program, often referred to as a "Hello World" app.

1. main() Function:

- This is the starting point of the Flutter app.
- The `runApp()` function takes the root widget (in this case, `MyApp`) and inflates it, making it the entire widget tree.

```
void main() {  
    runApp(MyApp());  
}
```

Mobile App Development

Creating Simple Flutter App



2. MyApp Class:

- This is a custom widget. Every Flutter app is built using widgets.

- In this example, `MyApp` extends `StatelessWidget` (indicating that the app doesn't maintain any mutable state).

```
class MyApp extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      // Define the home screen widget here  
      home: Scaffold(  
        appBar: AppBar(  

```

```
        appBar: AppBar(  
          title: Text('Flutter Basic Structure'),  
        ),  
        body: Center(  
          child: Text('Hello, Flutter!'),  
        ),  
      ),  
    );  
  }  
}
```

3. MaterialApp Widget:

- The **MaterialApp** widget wraps the entire app and provides various Material Design components like theme, routing, etc.
- It is a fundamental widget for Flutter apps following Material Design principles.

4. Scaffold Widget:

- **Scaffold** is a basic structure that provides an app's layout, including an app bar, body, and floating action button.
- It creates the framework for the visible elements of the app.

Mobile App Development

Creating Simple Flutter App



5. AppBar Widget:

- The `AppBar` widget is the top toolbar of the app, typically used to show a title or actions.

```
appBar: AppBar(  
  
  title: Text('Flutter Basic Structure'),  
  
)
```

Mobile App Development

Creating Simple Flutter App



6. Body:

- The `body` of the `Scaffold` widget contains the app's main content.
- In this example, we use the `Center` widget to center a `Text` widget that displays "Hello, Flutter!".

```
body: Center(  
  child: Text('Hello, Flutter!'),  
)
```

Let me break down the Flutter code you shared, step by step:

```
import 'package:flutter/material.dart';
```

Mobile App Development

Creating Simple Flutter App



```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(body: Center(child: Text('Hello,
World!'))),
    );
  }
}
```



THANK YOU

Dr. Annu Sharma
Associate Professor
Department of Computer Science